Fixed-parameter optimization of n-fold integer programs with large entries

Jana Cslovjecsek, Martin Koutecký, Alexandra Lassota, Michał Pilipczuk, Adam Polak

arxiv:2311.01890v2; CC BY 4.0

Editorial note. Only thm:nfold-main counts. Full author mathematics, graphics and source-local proofs retained. Publisher metadata adapted to article/natbib; native unchanged.

Allowed standard prerequisites (editorial).

- Standard graph algorithms and ETH cited by authors. All source-local proofs and necessary context retained.

Abstract

We study two classic variants of block-structured integer programming. Two-stage stochastic programs are integer programs of the form $\{A_i\mathrm{x} + D_i\mathrm{y}_i = \mathrm{b}_i$ for all $i = 1, \ldots, n\}$, where A_i and D_i are bounded-size matrices. Intuitively, this form corresponds to the setting when after setting a small set of global variables x, the program can be decomposed into a possibly large number of bounded-size subprograms. On the other hand, n-fold programs are integer programs of the form $\{\sum_{i=1}^{n} C_i\mathrm{y}_i = \mathrm{a}$ and $D_i\mathrm{y}_i = \mathrm{b}_i$ for all $i = 1, \ldots, n\}$, where again C_i and D_i are bounded-size matrices. This form is natural for knapsack-like problems, where we have a large number of variables partitioned into small-size groups, each group needs to obey some set of local constraints, and there are only a few global constraints that link together all the variables.

A line of recent work established that the optimization problem for both two-stage stochastic programs and n-fold programs is fixed-parameter tractable when parameterized by the dimensions of relevant matrices A_i, C_i, D_i and by the maximum absolute value of any entry appearing in the constraint matrix. A fundamental tool used in these advances is the notion of the Graver basis of a matrix, and this tool heavily relies on the assumption that all the entries of the constraint matrix are bounded.

In this work, we prove that the parameterized tractability results for two-stage stochastic and n-fold programs persist even when one allows large entries in the global part of the program. More precisely, we prove the following:

- The feasibility problem for two-stage stochastic programs is fixed-parameter tractable when parameterized by the dimensions of matrices A_i, D_i and by the maximum absolute value of the entries of matrices D_i. That is, we allow matrices A_i to have arbitrarily large entries.
- The linear optimization problem for n-fold integer programs that are uniform – all matrices C_i are equal – is fixed-parameter tractable when parameterized by the dimensions of matrices C_i and D_i and by the maximum absolute value of the entries of matrices D_i. That is, we require that $C_i = C$ for all $i = 1, \ldots, n$, but we allow C to have arbitrarily large entries.

In the second result, the uniformity assumption is necessary; otherwise the problem is NP-hard already when the parameters take constant values. Both our algorithms are weakly polynomial: the running time is measured in the total bitsize of the input.

In both results, we depart from the approach that relies purely on Graver bases. Instead, for two-stage stochastic programs, we devise a reduction to integer programming with a bounded number of variables using new insights about the combinatorics of integer cones. For n-fold programs, we reduce a given n-fold program to an exponential-size program with bounded right-hand sides, which we consequently solve using a reduction to mixed integer programming with a bounded number of integral variables. For n-fold programs, we reduce a given n-fold program to an exponential-size program with bounded right-hand sides, which we consequently solve using a reduction to mixed integer programming with a bounded number of integral variables.

1 Introduction

We study two variants of integer programming problems, where the specific structure of the constraint matrix can be exploited for the design of efficient parameterized algorithms. Two-stage stochastic programs are integer programs of the following form:

$$\begin{aligned} \mathrm{x} \in \mathbb{Z}^k_{\geqslant 0},\ \mathrm{y}_i \in \mathbb{Z}^k_{\geqslant 0}, \qquad & \text{and} \\ A_i\mathrm{x} + D_i\mathrm{y}_i = \mathrm{b}_i \qquad & \text{for all } i = 1, 2, \ldots, n. \end{aligned} \tag{♠}$$

Here, A_i, D_i are integer $k \times k$ matrices[1] and each b_i is an integer vector of length k. This form arises naturally when the given integer program can be decomposed into multiple independent subprograms on disjoint variable sets y_i, except there are several global variables x that are involved in all the subprograms and thus link them. See the survey of Shultz et al. [47] as well as an exposition article by Gavenčiak et al. [22] for examples of applications.

[1]Reliance on square matrices is just for convenience of presentation. The setting where blocks are rectangular matrices with dimensions bounded by k can be reduced to the setting of $k \times k$ square matrices by just padding with 0s.

We moreover study n-fold programs which are integer programs of the form

$$\begin{aligned} & \mathrm{y}_i \in \mathbb{Z}^k_{\geqslant 0}, & \\ \sum_{i=1}^{n} & C_i \mathrm{y}_i = \mathrm{a}, & \text{and} \qquad (\clubsuit) \\ & D_i \mathrm{y}_i = \mathrm{b}_i & \text{for all } i = 1, 2, \ldots, n, \end{aligned}$$

where again C_i, D_i are integer $k \times k$ matrices and a, b_i are integer vectors of length k. This kind of programs appears for knapsack-like and scheduling problems, where blocks of variables y_i correspond to some independent local decisions (for instance, whether to pack an item into the knapsack or not), and there are only a few linear constraints that involve all variables (for instance, that the capacity of the knapsack is not exceeded). See [10, 17, 22, 26, 31, 33, 34, 36] for examples of n-fold programs appearing naturally "in the wild". Figure 1 depicts constraint matrices of two-stage stochastic and n-fold programs.

$$\begin{bmatrix} A_1 & D_1 & & & \\ A_2 & & D_2 & & \\ \vdots & & & \ddots & \\ A_n & & & & D_n \end{bmatrix} \qquad \begin{bmatrix} C_1 & C_2 & \ldots & C_n \\ D_1 & & & \\ & D_2 & & \\ & & \ddots & \\ & & & D_n \end{bmatrix} \qquad \begin{bmatrix} B & C_1 & C_2 & \ldots & C_n \\ A_1 & D_1 & & & \\ A_2 & & D_2 & & \\ \vdots & & & \ddots & \\ A_n & & & & D_n \end{bmatrix}$$

Figure 1: Constraint matrices in two-stage stochastic, n-fold, and 4-block integer programs, respectively. (The last kind will be discussed later.) Every block is a $k \times k$ matrix, where k is the parameter. Empty spaces denote blocks of zeroes.

Both for two-stage stochastic programs and for n-fold programs, we can consider two computational problems. The simpler feasibility problem just asks whether the given program has a solution: an evaluation of variables in nonnegative integers that satisfies all the constraints. In the harder (linear) optimization problem, we are additionally given an integer weight w_x for every variable x appearing in the program, and the task is to minimize $\sum_{x\colon \text{variable}} w_x \cdot x$ over all solutions.

Two-stage stochastic and n-fold programs have recently gathered significant interest in the theoretical community for two reasons. On one hand, it turns out that for multiple combinatorial problems, their natural integer programming formulations take either of the two forms. On the other hand, one can actually design highly non-trivial fixed-parameter algorithms for the optimization problem for both two-stage stochastic and n-fold programs; we will review them in a minute. Combining this two points yields a powerful algorithmic technique that allowed multiple new tractability results and running times improvements for various problems of combinatorial optimization; see [10, 22, 26, 31–34, 36, 40] for examples.

Delving more into technical details, if by Δ we denote the maximum absolute value of any entry in the constraint matrix, then the optimization problem for

- two-stage stochastic programs (♠) can be solved in time $2^{\Delta^{\mathcal{O}(k^2)}} \cdot n \log^{\mathcal{O}(k^2)} n$ [13]; and
- n-fold programs (♣) can be solved in time $(k\Delta)^{\mathcal{O}(k^3)} \cdot n \log^{\mathcal{O}(k^2)} n$ [12].

The results above are in fact pinnacles of an over-a-decade-long sequence of developments, which gradually improved both the generality of the results and the running times [3, 12, 13, 17, 19, 20, 23, 25, 29, 41], as well as provided complexity lower bounds [27, 35]. We refer the interested reader to the monumental manuscript of Eisenbrand et al. [20] which provides a comprehensive perspective on this research area.

We remark that the tractability results presented above can be also extended to the setting where the global-local block structure present in two-stage stochastic and n-fold programs can be iterated further, roughly speaking to trees of bounded depth. This leads to the study of integer programs with bounded primal or dual treedepth, for which analogous tractability results have been established. Since these notions will not be of interest in this work, we refrain from providing further details and refer the interested reader to the works relevant for this direction [5, 6, 9, 12, 13, 19, 20, 29, 30, 35, 41].

All the above-mentioned works, be it on two-stage stochastic or n-fold programs, or on programs of bounded primal or dual treedepth, crucially rely on one tool: the notion of the Graver basis of a matrix. Formal definition can be found in Section 5.1, but intuitively speaking, the Graver basis of a matrix A consists of "minimal steps" within the lattice of integer points belonging to the kernel of A. Therefore, the maximum norm of an element of the Graver basis reflects the "granularity" of this lattice. And so, the two fundamental observations underlying all the discussed developments are the following:

- in two-stage stochastic matrices (or more generally, matrices of bounded primal treedepth), the ℓ_∞ norm of the elements of the Graver basis is bounded in terms of k (the dimension of every block) and the maximum absolute value of any entry appearing in the matrix (see [20, Lemma 28]); and
- an analogous result holds for n-fold matrices (or more generally, matrices of bounded dual treedepth) and the ℓ_1 norm (see [20, Lemma 30]).

Based on these observations, various algorithmic strategies, including augmentation frameworks [25, 41] and proximity arguments [12, 13, 19], can be employed to solve respective integer programs.

The drawback of the Graver-based approach is that it heavily relies on the assumption that all the entries of the input matrices are (parametrically) bounded. Indeed, the norms of the elements of the Graver basis are typically at least as large as the entries, so lacking any upper bound on the latter renders the methodology inapplicable. This is in stark contrast with the results on fixed-parameter tractability of integer programming parameterized by the number of variables [14, 15, 21, 28, 43, 46], which rely on different tools and for which no bound on the absolute values of the entries is required. In fact, two-stage stochastic programs generalize programs with a bounded number of variables (just do not use variables y_i), yet the current results for two-stage stochastic pro-

grams do not generalize those for integer programs with few variables, because they additionally assume a bound on the absolute values of the entries.

The goal of this paper is to understand to what extent one can efficiently solve two-stage stochastic and n-fold programs while allowing large entries on input.

Our contribution. We prove that both the feasibility problem for two-stage stochastic programs and the optimization problem for uniform n-fold programs (that is, where $C_1 = C_2 = \ldots = C_n$) can be solved in fixed-parameter time when parameterized by the dimensions of the blocks and the maximum absolute value of any entry appearing in the diagonal blocks D_i. That is, we allow the entries of the global blocks (A_i and C_i, respectively) to be arbitrarily large, and in the case of n-fold programs, we require that all blocks C_i are equal. The statements below summarize our results. ($\|P\|$ denotes the total bitsize of a program P.)

Theorem 1. The feasibility problem for two-stage stochastic programs P of the form (♠) can be solved in time $f(k, \max_i \|D_i\|_\infty) \cdot \|P\|$ for a computable function f, where k is the dimension of all the blocks A_i, D_i.

Theorem 2. The optimization problem for n-fold programs P of the form (♣) that are uniform (that is, satisfy $C_1 = \ldots = C_n$) can be solved in time $f(k, \max_i \|D_i\|_\infty) \cdot \|P\|^{\mathcal{O}(1)}$ for a computable function f, where k is the dimension of all the blocks C_i, D_i.

We remark that in Theorem 1 it is necessary to include $\max_i \|D_i\|_\infty$ among the parameters, because with no bound on the entries of the constraint matrix, the feasibility problem for two-stage stochastic programs becomes NP-hard already for a constant k. An easy way to see it is via a straightforward reduction from an NP-hard problem called Good Simultaneous Approximation [42], where the input consists of a rational vector $\mathrm{a} \in \mathbb{Q}^n$, a desired error bound $\epsilon \in \mathbb{Q}$, and an upper bound $N \in \mathbb{Z}_{\geqslant 0}$, and the goal is to decide if there exists an integer multiplier $Q \in \{1, 2, \ldots, N\}$ such that $Q\mathrm{a}$ is ϵ-close in the infinity norm to an integer vector, i.e., $\exists_{\mathrm{b} \in \mathbb{Z}^n} \|Q\mathrm{a} - \mathrm{b}\|_\infty \leqslant \epsilon$. In Section B we give a different argument, reducing directly from 3-SAT, and showing that two-stage stochastic feasibility is actually strongly NP-hard for $k = 16$. This rules out also any running time of the form $f(k) \cdot (\|P\| + \max_i \|D_i\|_\infty)^{\mathcal{O}(1)}$.

The uniformity condition in Theorem 2 is also necessary (unless P = NP), as one can very easily reduce Subset Sum to the feasibility problem for n-fold programs with $k = 2$ and D_i being $\{0, 1\}$-matrices. Indeed, given an instance of Subset Sum consisting of positive integers $a_1, \ldots, a_n$ and a target integer t, we can write the following n-fold program on variables $y_1, \ldots, y_n, y'_1, \ldots, y'_n \in \mathbb{Z}_{\geqslant 0}$: $y_i + y'_i = 1$ for all $i = 1, \ldots, n$ and $\sum_{i=1}^n a_i y_i = t$. We remark that uniform n-fold programs are actually of the highest importance, as this form typically arises in applications. In fact, many of the previous works named such problems "n-fold", while our formulation (♣) would be called "generalized n-fold".

We also remark that the algorithm of Theorem 2 does not use the assumption that the number of rows of matrix C is bounded by k (formally, in the precise,

statement Theorem 24, we do not consider this number among parameters). However, we stress that the bound on the number of columns of C is heavily exploited, which sets our approach apart from many of the previous works [12, 19, 41].

Further, observe that Theorem 1 applies only to the feasibility problem for two-stage stochastic programs. We actually do not know whether Theorem 1 can be extended to the optimization problem as well, and we consider determining this an outstanding open problem. We will discuss its motivation in more details later on. Also, we remark that Theorem 1 seems to be the first algorithm for feasibility of two-stage stochastic programs that achieves truly linear dependence of the running time on the total input size; the earlier algorithms of [13, 41] had at least some additional polylogarithmic factors.

Finally, note that the algorithms of Theorems 1 and 2 are not strongly polynomial (i.e., the running time depends on the total bitsize of the input, rather than is counted in the number of arithmetic operations), while the previous algorithms of [12, 13, 19, 41] for the stronger parameterization are. At least in the case of Theorem 1, this is justified, as the problem considered there generalizes integer programming parameterized by the number of variables, for which strongly polynomial FPT algorithms are not known.

Not surprisingly, the proofs of Theorems 1 and 2 depart from the by now standard approach through Graver bases; they are based on entirely new techniques, with some key Graver-based insight needed in the case of Theorem 2. In both cases, the problem is ultimately reduced to (mixed) integer programming with a bounded number of (integral) variables, and this allows us to cope with large entries on input. We expand the discussion of our techniques in Section 2, which contains a technical overview of the proofs.

4-block programs. Finally, we would like to discuss another motivation for investigating two-stage stochastic and n-fold programs with large entries, namely the open question about the parameterized complexity of 4-block integer programming. 4-block programs are programs in which the constraint matrix has the block-structured form depicted in the right panel of Figure 1; note that this form naturally generalizes both two-stage stochastic and n-fold programs. It is an important problem in the area to determine whether the feasibility problem for 4-block programs can be solved in fixed-parameter time when parameterized by the dimension of blocks k and the maximum absolute value of any entry in the input matrix. The question was asked by Hemmecke et al. [24], who proposed an XP algorithm for the problem. Improvements on the XP running time were reported by Chen et al. [11], and FPT algorithms for special cases were proposed by Chen et al. [8]; yet no FPT algorithm for the problem in full generality is known so far. We remark that recently, Chen et al. [7] studied the complexity of 4-block programming while allowing large entries in all the four blocks of the matrix. They showed that then the problem becomes NP-hard already for blocks of constant dimension, and they discussed a few special cases that lead to tractability.

We observe that in the context of the feasibility problem for uniform 4-block programs (i.e., with $A_i = A$ and $C_i = C$ for all $i = 1, \ldots, n$), it is possible to emulate large entries within the global blocks A, B, C using only small entries at the cost of adding a bounded number of auxiliary variables. This yields the following reduction, which we prove in Section A.

Observation 3. Suppose the feasibility problem for uniform 4-block programs can be solved in time $f(k, \Delta) \cdot \|P\|^{\mathcal{O}(1)}$ for some computable function f, where k is the dimension of every block and Δ is the maximum absolute value of any entry in the constraint matrix. Then the feasibility problem for uniform 4-block programs can be also solved in time $g(k, \max_i \|D_i\|_\infty) \cdot \|P\|^{\mathcal{O}(1)}$ for some computable function g under the assumption that all the absolute values of the entries in matrices A, B, C are bounded by n.

Consequently, to approach the problem of fixed-parameter tractability of 4-block integer programming, it is imperative to understand first the complexity of two-stage stochastic and n-fold programming with large entries allowed in the global blocks. And this is precisely what we do in this work.

We believe that the next natural step towards understanding the complexity of 4-block integer programming would be to extend Theorem 1 to the optimization problem; that is, to determine whether optimization of two-stage stochastic programs can be solved in fixed-parameter time when parameterized by k and $\max_i \|D_i\|_\infty$. Indeed, lifting the result from feasibility to the optimization problem roughly corresponds to adding a single constraint that links all the variables, and 4-block programs differ from two-stage stochastic programs precisely in that there may be up to k such additional linking constraints. Thus, we hope that the new approach to block-structured integer programming presented in this work may pave the way towards understanding the complexity of solving 4-block integer programs.

Acknowledgements. This research has been initiated during the trimester on Discrete Optimization at the Hausdorff Research Institute for Mathematics (HIM) in Bonn, Germany. We thank the organizers of the trimester for creating a friendly and motivating research environment. We also thank Eleonore Bach, Fritz Eisenbrand, and Robert Weismantel, for pointing us to the work of Aliev and Henk [2]. Finally, we thank an anonymous reviewer for suggesting a simpler proof of Lemma 29 as well as pointing out the possibility of obtaining the lower bound presented in Section B.

2 Overview

In this section, we provide a technical overview of our results aimed at presenting the main ideas and new conceptual contributions.

2.1 Two-stage stochastic programming

We start with an overview on the proof of Theorem 1. We will heavily rely on the combinatorics of integer and polyhedral cones, so let us recall basic definitions and properties.

Cones. Consider an integer matrix D with t columns and k rows. The polyhedral cone spanned by D is the set $\mathsf{cone}(D) \coloneqq \{D\mathrm{y} \colon \mathrm{y} \in \mathbb{R}^t_{\geqslant 0}\} \subseteq \mathbb{R}^k_{\geqslant 0}$, or equivalently, the set of all vectors in $\mathbb{R}^k_{\geqslant 0}$ expressible as nonnegative combinations of the columns of D. Within the polyhedral cone, we have the integer cone where we restrict attention to nonnegative integer combinations: $\mathsf{intCone}(D) \coloneqq \{D\mathrm{y} \colon \mathrm{y} \in \mathbb{Z}^t_{\geqslant 0}\} \subseteq \mathbb{Z}^k$. Finally, the integer lattice is the set $\mathsf{lattice}(D) \coloneqq \{D\mathrm{y} \colon \mathrm{y} \in \mathbb{Z}^t\} \subseteq \mathbb{Z}^k$ which comprises all integer combinations of columns of D with possibly negative coefficients.

Clearly, not every integer vector in $\mathsf{cone}(D)$ has to belong to $\mathsf{intCone}(D)$. It is not even necessarily the case that $\mathsf{intCone}(D) = \mathsf{cone}(D) \cap \mathsf{lattice}(D)$, as there might be vectors that can be obtained both as a nonnegative combination and as an integer combination of columns of D, but every such integer combination must necessarily contain negative coefficients. To see an example, note that in dimension $k = 1$, this is the Frobenius problem: supposing all entries of D are positive integers, the elements of $\mathsf{intCone}(D)$ are essentially all nonnegative numbers divisible by the gcd (greatest common divisor) of the entries of D, except that for small numbers there might be some aberrations: a positive integer of order $\mathcal{O}(\|D\|^2_\infty)$ may not be presentable as a nonnegative combination of the entries of D, even assuming it is divisible by the gcd of the entries of D.

However, the Frobenius example suggests that the equality $\mathsf{intCone}(D) = \mathsf{cone}(D) \cap \mathsf{lattice}(D)$ is almost true, except for aberrations near the boundary of $\mathsf{cone}(D)$. We forge this intuition into a formal statement presented below that says roughly the following: if one takes a look at $\mathsf{intCone}(D)$ at a large scale, by restricting attention to integer vectors $\mathrm{v} \in \mathbb{Z}^k$ with fixed remainders of entries modulo some large integer B, then $\mathsf{intCone}(D)$ behaves like a polyhedron. In the following, for a positive integer B and a vector $\mathrm{r} \in \{0, 1, \ldots, B-1\}^k$, we let Λ^B_r be the set of all vectors $\mathrm{v} \in \mathbb{Z}^k$ such that $\mathrm{v} \equiv \mathrm{r} \bmod B$, which means $v_i \equiv r_i \bmod B$ for all $i \in \{1, \ldots, k\}$.

Theorem 4 (Reduction to Polyhedral Constraints, see Theorem 19). Let D be an integer matrix with t columns and k rows. Then there exists a positive integer B, computable from D, such that for every $\mathrm{r} \in \{0, 1, \ldots, B-1\}^k$, there exists a polyhedron $\mathcal{Q}_\mathrm{r}$ such that

$$\Lambda^B_\mathrm{r} \cap \mathsf{intCone}(D) = \Lambda^B_\mathrm{r} \cap \mathcal{Q}_\mathrm{r}.$$

Moreover, a representation of such a polyhedron $\mathcal{Q}_\mathrm{r}$ can be computed given D and r.

In other words, Theorem 4 states that if one fixes the remainders of entries modulo B, then membership in the integer cone can be equivalently expressed

through a finite system of linear inequalities. Before we sketch the proof of Theorem 4, let us discuss how to use this to solve two-stage stochastic programs.

The algorithm. Consider a two-stage stochastic program $P = (A_i, D_i, \mathrm{b}_i \colon i \in \{1, \ldots, n\})$ such that blocks A_i, D_i are integer $k \times k$ matrices and all entries of blocks D_i are bounded in absolute value by Δ. The feasibility problem for P can be understood as the question about satisfaction of the following sentence, where all quantifications range over $\mathbb{Z}^k_{\geqslant 0}$:

$$\exists_{\mathrm{x}} \left(\bigwedge_{i=1}^{n} \exists_{\mathrm{y}_i} \; A_i\mathrm{x} + D_i\mathrm{y}_i = \mathrm{b}_i \right), \quad \text{or equivalently,} \quad \exists_{\mathrm{x}} \left(\bigwedge_{i=1}^{n} \; \mathrm{b}_i - A_i\mathrm{x} \in \mathsf{intCone}(D_i) \right). \tag{1}$$

Applying Theorem 4 to each matrix D_i yields a positive integer B_i. Note that there are only at most $(2\Delta + 1)^{k^2}$ different matrices D_i appearing in P, which also bounds the number of different integers B_i. By replacing all B_is with their least common multiple, we may assume that $B_1 = B_2 = \ldots = B_n = B$. Note that B is bounded by a computable function of Δ and k.

Consider a hypothetical solution $\mathrm{x}, (\mathrm{y}_i \colon i \in \{1, \ldots, n\})$ to P. We guess, by branching into B^k possibilities, a vector $\mathrm{r} \in \{0, 1, \ldots, B-1\}^k$ such that $\mathrm{x} \equiv \mathrm{r} \bmod B$. Having fixed r, we know how the vectors $\mathrm{b}_i - A_i\mathrm{x}$ look like modulo B, hence by Theorem 4, we may replace the assertion $\mathrm{b}_i - A_i\mathrm{x} \in \mathsf{intCone}(D_i)$ with the assertion $\mathrm{b}_i - A_i\mathrm{x} \in \mathcal{Q}_{\mathrm{r}_i}$, where $\mathrm{r}_i \in \{0, 1, \ldots, B-1\}^k$ is the unique vector such that $\mathrm{b}_i - A_i\mathrm{r} \equiv \mathrm{r}_i \bmod B$. Thus, (1) can be rewritten to the sentence

$$\bigvee_{\mathrm{r} \in \{0,1,\ldots,B-1\}^k} \exists_{\mathrm{x}} \; (\mathrm{x} \equiv \mathrm{r} \bmod B) \wedge \left(\bigwedge_{i=1}^{n} \mathrm{b}_i - A_i\mathrm{x} \in \mathcal{Q}_{\mathrm{r}_i} \right),$$

which is equivalent to

$$\bigvee_{\mathrm{r} \in \{0,1,\ldots,B-1\}^k} \exists_{\mathrm{x}} \exists_{\mathrm{z}} \; (\mathrm{x} = B \cdot \mathrm{z} + \mathrm{r}) \wedge \left(\bigwedge_{i=1}^{n} \mathrm{b}_i - A_i\mathrm{x} \in \mathcal{Q}_{\mathrm{r}_i} \right). \tag{2}$$

Verifying satisfiability of (2) boils down to solving B^k integer programs on $2k$ variables x and z and linearly FPT many constraints, which can be done in linear fixed-parameter time using standard algorithms, for instance that of Kannan [28].

We remark that the explanation presented above highlights that Theorem 4 can be understood as a quantifier elimination result in the arithmetic theory of integers. This may be of independent interest, but we do not pursue this direction in this work.

Reduction to polyhedral constraints. We are left with sketching the proof of Theorem 4. Let $\mathcal{Z} \coloneqq \Lambda^B_{\mathrm{r}} \cap \mathsf{intCone}(D)$. Our goal is to understand that $\mathcal{Z}$ can be expressed as the points of Λ^B_{r} that are contained in some polyhedron $\mathcal{Q} = \mathcal{Q}_{\mathrm{r}}$.

The first step is to understand $\mathsf{cone}(D)$ itself as a polyhedron. This understanding is provided by a classic theorem of Weyl [48]: given D, one can compute a set of integer vectors $\mathcal{F} \subseteq \mathbb{Z}^k$ such that

$$\mathsf{cone}(D) = \{\mathrm{v} \in \mathbb{R}^k \mid \langle \mathrm{f}, \mathrm{v} \rangle \geqslant 0 \text{ for all } \mathrm{f} \in \mathcal{F}\}.$$

Here, $\langle \cdot, \cdot \rangle$ denotes the scalar product in $\mathbb{R}^k$. We will identify vectors $\mathrm{f} \in \mathcal{F}$ with their associated linear functionals $\mathrm{v} \mapsto \langle \mathrm{f}, \mathrm{v} \rangle$. Thus, $\mathsf{cone}(D)$ comprises all vectors v that have nonnegative evaluations on all functionals in $\mathcal{F}$. It is instructive to also think of the elements of $\mathcal{F}$ as of the facets of $\mathsf{cone}(D)$ understood as a polyhedron, where the functional associated with $\mathrm{f} \in \mathcal{F}$ measures the distance from the corresponding facet.

Recall that in the context of Theorem 4, we consider vectors of Λ_{r}^B, that is, vectors $\mathrm{v} \in \mathbb{Z}^k$ such that $\mathrm{v} \equiv \mathrm{r} \bmod B$. Then $\langle \mathrm{f}, \mathrm{v} \rangle \equiv \langle \mathrm{f}, \mathrm{r} \rangle \bmod B$ for every $\mathrm{f} \in \mathcal{F}$, hence we can find a unique integer $p_{\mathrm{f}} \in \{0, 1, \ldots, B-1\}$, $p_{\mathrm{f}} \equiv \langle \mathrm{f}, \mathrm{r} \rangle \bmod B$, such that $\langle \mathrm{f}, \mathrm{v} \rangle \equiv p_{\mathrm{f}} \bmod B$ for all $\mathrm{v} \in \Lambda_{\mathrm{r}}^B$. Now $\langle \mathrm{f}, \mathrm{v} \rangle$ is also nonnegative provided $\mathrm{v} \in \mathsf{cone}(D)$, hence

$$\langle \mathrm{f}, \mathrm{v} \rangle \in \{p_{\mathrm{f}}, p_{\mathrm{f}} + B, p_{\mathrm{f}} + 2B, \ldots\} \qquad \text{for all } \mathrm{f} \in \mathcal{F} \text{ and } \mathrm{v} \in \Lambda_{\mathrm{r}}^B \cap \mathsf{cone}(D).$$

Now comes the key distinction about the behavior of $\mathrm{v} \in \Lambda_{\mathrm{r}}^B \cap \mathsf{cone}(D)$ with respect to $\mathrm{f} \in \mathcal{F}$: we say that f is tight with respect to v if $\langle \mathrm{f}, \mathrm{v} \rangle = p_{\mathrm{f}}$, and is not tight otherwise, that is, if $\langle \mathrm{f}, \mathrm{v} \rangle \geqslant p_{\mathrm{f}} + B$. Recall that in the context of Theorem 4, we are eventually free to choose B to be large enough. Intuitively, this means that if f is not tight for v, then v lies far from the facet corresponding to f and there is a very large slack in the constraint posed by f understood as a functional. On the other hand, if f is tight with respect to v, then v is close to the boundary of $\mathsf{cone}(D)$ at the facet corresponding to f, and there is a potential danger of observing Frobenius-like aberrations at v.

Thus, the set $\mathcal{R} \coloneqq \Lambda_{\mathrm{r}}^B \cap \mathsf{cone}(D)$ can be partitioned into subsets $\{\mathcal{R}_{\mathcal{G}} \colon \mathcal{G} \subseteq \mathcal{F}\}$ defined as follows: $\mathcal{R}_{\mathcal{G}}$ comprises all vectors $\mathrm{v} \in \mathcal{R}$ such that $\mathcal{G}$ is exactly the set of functionals $\mathrm{f} \in \mathcal{F}$ that are tight with respect to v. Our goal is to prove that each set $\mathcal{R}_{\mathcal{G}}$ behaves uniformly with respect to $\mathcal{Z}$: it is either completely disjoint or completely contained in $\mathcal{Z}$. To start the discussion, let us look at the particular case of $\mathcal{R}_{\mathcal{G}}$ for $\mathcal{G} = \emptyset$. These are vectors that are deep inside $\mathsf{cone}(D)$, for which no functional in $\mathcal{F}$ is tight. For these vectors, we use the following lemma, which is the cornerstone of our proof.

Lemma 5 (Deep-in-the-Cone Lemma, simplified version of Lemma 16). There exists a constant M, depending only on D, such that the following holds. Suppose $\mathrm{v} \in \mathsf{cone}(D) \cap \mathbb{Z}^k$ is such that $\langle \mathrm{f}, \mathrm{v} \rangle > M$ for all $\mathrm{f} \in \mathcal{F}$. Then $\mathrm{v} \in \mathsf{intCone}(D)$ if and only if $\mathrm{v} \in \mathsf{lattice}(D)$.

Proof. The left-to-right implication is obvious, hence let us focus on the right-to-left implication. Suppose then that $\mathrm{v} \in \mathsf{lattice}(D)$.

Let $\mathrm{w} = \sum_{\mathrm{d} \in D} L \cdot \mathrm{d}$, where the summation is over the columns of D and L is a positive integer to be fixed later. Observe that for every $\mathrm{f} \in \mathcal{F}$, we have

$\langle \mathrm{f}, \mathrm{v} - \mathrm{w} \rangle > M - L \cdot \sum_{\mathrm{d} \in D} \langle \mathrm{f}, \mathrm{d} \rangle$. Therefore, if we choose M to be not smaller than $L \cdot \max_{\mathrm{f} \in \mathcal{F}} \|\mathrm{f}\|_1 \cdot \|D\|_\infty$, then we are certain that $\langle \mathrm{f}, \mathrm{v} - \mathrm{w} \rangle \geqslant 0$ for all $\mathrm{f} \in \mathcal{F}$, and hence $\mathrm{v} - \mathrm{w} \in \mathsf{cone}(D)$. Consequently, we can write $\mathrm{v} - \mathrm{w} = D\mathrm{y}$ for some $\mathrm{y} \in \mathbb{R}^t_{\geqslant 0}$. Let $\mathrm{y}' \in \mathbb{Z}^t_{\geqslant 0}$ be such that $y'_i = \lfloor y_i \rfloor$ for all $i \in \{1, \ldots, t\}$, and let $\mathrm{v}' = \mathrm{w} + D\mathrm{y}'$. Then

$$\|\mathrm{v} - \mathrm{v}'\|_\infty = \|D(\mathrm{y} - \mathrm{y}')\|_\infty \leqslant t \cdot \|D\|_\infty.$$

On the other hand, we clearly have $\mathrm{v}' \in \mathsf{intCone}(D)$ and by assumption, $\mathrm{v} \in \mathsf{lattice}(D)$. It follows that $\mathrm{v} - \mathrm{v}' \in \mathsf{lattice}(D)$. From standard bounds, see e.g. [44], it follows that there exists $\mathrm{z} \in \mathbb{Z}^t$ with $\mathrm{v} - \mathrm{v}' = D\mathrm{z}$ such that $\|\mathrm{z}\|_1$ is bounded by a function of D and $\|\mathrm{v} - \mathrm{v}'\|_\infty$, which in turn is again bounded by a function of D as explained above. (Note here that t is the number of columns of D, hence it also depends only on D.) This means that if we choose L large enough depending on D, we are certain that $\|\mathrm{z}\|_1 \leqslant L$. Now, it remains to observe that

$$\mathrm{v} = \mathrm{w} + D\mathrm{y}' + (\mathrm{v} - \mathrm{v}') = D(L \cdot \mathbf{1} + \mathrm{y}' + \mathrm{z}),$$

where $\mathbf{1}$ denotes the vector of t ones, and that all the entries of $L \cdot \mathbf{1} + \mathrm{y}' + \mathrm{z}$ are nonnegative integers. This proves that $\mathrm{v} \in \mathsf{intCone}(D)$. □

We remark that the statement of Lemma 5 actually follows from results present in the literature, concerning the notion of diagonal Frobenius numbers. See the work of Aliev and Henk [2] for a broader discussion and pointers to earlier works, as well as the works of Aggrawal et al. [1] and of Bach et al. [4] for the newest developments on optimal bounds. As we will discuss in a moment, in this work we actually use a generalization of Lemma 5.

Consider any $\mathrm{u}, \mathrm{v} \in \mathcal{R}$. Since all the entries of $\mathrm{u} - \mathrm{v}$ are divisible by B, it is not hard to prove the following: if we choose B to be a large enough factorial, then $\mathrm{u} \in \mathsf{lattice}(D)$ if and only if $\mathrm{v} \in \mathsf{lattice}(D)$. Hence, from Lemma 5 it follows that $\mathcal{R}_\emptyset$ is either entirely disjoint or entirely contained in $\mathcal{Z}$.

A more involved reasoning based on the same fundamental ideas, but using a generalization of Lemma 5, yields the following lemma, which tackles also the case when some functionals of $\mathcal{F}$ are tight with respect to the considered vectors.

Lemma 6 (see Claim 21). Suppose $\mathrm{u}, \mathrm{v} \in \mathcal{R}$ are such that for every $\mathrm{f} \in \mathcal{F}$, if f is tight with respect to u, then f is also tight with respect to v. Then $\mathrm{u} \in \mathcal{Z}$ implies $\mathrm{v} \in \mathcal{Z}$.

We remark that the proof of Lemma 6 actually requires more work and more ideas than those presented in the proof of Lemma 5. In essence, one needs to partition functionals that are tight with respect to u into those that are very tight (have very small p_f) and those that are only slightly tight (have relatively large p_f) in order to create a sufficient gap between very tight and slightly tight functionals. Having achieved this, a delicate variant of the reasoning from the proof of Lemma 5 can be applied. It is important that whenever a functional $\mathrm{f} \in \mathcal{F}$ is tight with respect to both u and v, we actually know that $\langle \mathrm{f}, \mathrm{u} \rangle =$

$\langle \mathrm{f}, \mathrm{v} \rangle = p_{\mathrm{f}}$. Note that this is exactly the benefit achieved by restricting attention to the vectors of Λ_{r}^B.

Using Lemma 6, we can immediately describe how the structure of $\mathcal{Z}$ relates to that of $\mathcal{R}$.

Corollary 7 (see Claim 22). For every $\mathcal{G} \subseteq \mathcal{F}$, either $\mathcal{R}_{\mathcal{G}} \cap \mathcal{Z} = \emptyset$ or $\mathcal{R}_{\mathcal{G}} \subseteq \mathcal{Z}$. Moreover, if $\mathcal{R}_{\mathcal{G}} \subseteq \mathcal{Z}$ and $\mathcal{R}_{\mathcal{G}}$ is non-empty, then $\mathcal{R}_{\mathcal{G}'} \subseteq \mathcal{Z}$ for all $\mathcal{G}' \subseteq \mathcal{G}$.

Corollary 7 suggests now how to define the polyhedron $\mathcal{Q}$. Namely, $\mathcal{Q}$ is defined as the set of all $\mathrm{v} \in \mathbb{R}^k$ satisfying the following linear inequalities:

- inequalities $\langle \mathrm{f}, \mathrm{v} \rangle \geqslant 0$ for all $\mathrm{f} \in \mathcal{F}$ that define $\mathsf{cone}(D)$; and
- for every $\mathcal{G} \subseteq \mathcal{F}$ such that $\mathcal{R}_{\mathcal{G}}$ is nonempty and $\mathcal{R}_{\mathcal{G}} \cap \mathcal{Z} = \emptyset$, the inequality

$$\sum_{\mathrm{g} \in \mathcal{G}} \langle \mathrm{g}, \mathrm{v} \rangle \geqslant 1 + \sum_{\mathrm{g} \in \mathcal{G}} p_{\mathrm{g}}.$$

In essence, the inequalities from the second point "carve out" those parts $\mathcal{R}_{\mathcal{G}}$ that should not be included in $\mathcal{Z}$. We note that computing the inequalities defining $\mathcal{Q}$ requires solving several auxiliary integer programs to figure out for which $\mathcal{G} \subseteq \mathcal{F}$ the corresponding inequality should be included.

It is now straightforward to verify, using all the accumulated observations, that indeed $\mathcal{Z} = \mathcal{R} \cap \mathcal{Q}$ as required. This concludes a sketch of the proof of Theorem 4.

2.2 n-fold programming

We now give an overview of the proof of Theorem 2. For simplicity, we make the following assumptions.

- We focus on the feasibility problem instead of optimization. At the very end, we will remark on what additional ideas are needed to also tackle the optimization problem.
- We assume that all the diagonal blocks D_i are equal: $D_i = D$ for all $i \in \{1, \ldots, n\}$, where D is a $k \times k$ integer matrix with $\|D\|_\infty \leqslant \Delta$. This is only a minor simplification because there are only $(2\Delta + 1)^{k^2}$ different matrices D_i with $\|D_i\|_\infty \leqslant \Delta$, and in the general case, we simply treat every such possible matrix "type" separately using the reasoning from the simplified case.

Breaking up bricks. Basic components of the given n-fold program $P = (C, D, \mathrm{a}, \mathrm{b}_i : i \in \{1, \ldots, n\})$ are bricks: programs $D\mathrm{y}_i = \mathrm{b}_i$ for $i \in \{1, \ldots, n\}$ that encode local constraints on the variables y_i. While the entries of D are bounded in absolute values by the parameter Δ, we do not assume any bound on the entries of vectors b_i. This poses an issue, as different bricks may have very different behaviors.

The key idea in our approach is to simplify the program P by iteratively breaking up every brick $D\mathrm{y} = \mathrm{b}$ into two bricks $D\mathrm{y} = \mathrm{b}'$ and $D\mathrm{y} = \mathrm{b}''$ with strictly smaller right-hand sides $\mathrm{b}', \mathrm{b}''$, until eventually, we obtain an equivalent

n-fold program P' in which all right-hand sides have ℓ_∞-norms bounded in terms of the parameters. The following lemma is the crucial new piece of technology used in our proof. (Here, we use the conformal order on $\mathbb{Z}^k$: we write $\mathrm{u} \sqsubseteq \mathrm{v}$ if $|\mathrm{u}[i]| \leqslant |\mathrm{v}[i]|$ and $\mathrm{u}[i] \cdot \mathrm{v}[i] \geqslant 0$ for all $i \in \{1, \ldots, k\}$.)

Lemma 8 (Brick Decomposition Lemma, see Lemma 27). There exists a function $g(k, \Delta) \in 2^{(k\Delta)^{\mathcal{O}(k)}}$ such that the following holds. Let D be an integer matrix with t columns and k rows and all absolute values of its entries bounded by Δ. Further, let $\mathrm{b} \in \mathbb{Z}^k$ be an integer vector such that $\|\mathrm{b}\|_\infty > g(k, \Delta)$. Then there are non-zero vectors $\mathrm{b}', \mathrm{b}'' \in \mathbb{Z}^k$ such that:

- $\mathrm{b}', \mathrm{b}'' \sqsubseteq \mathrm{b}$ and $\mathrm{b} = \mathrm{b}' + \mathrm{b}''$; and
- for every $\mathrm{v} \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ satisfying $D\mathrm{v} = \mathrm{b}$, there exist $\mathrm{v}', \mathrm{v}'' \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ such that

$$\mathrm{v} = \mathrm{v}' + \mathrm{v}'', \qquad D\mathrm{v}' = \mathrm{b}', \qquad \text{and} \qquad D\mathrm{v}'' = \mathrm{b}''.$$

In other words, Lemma 8 states that the brick $D\mathrm{y} = \mathrm{b}$ can be broken into two new bricks $D\mathrm{y}' = \mathrm{b}'$ and $D\mathrm{y}'' = \mathrm{b}''$ with conformally strictly smaller $\mathrm{b}', \mathrm{b}''$ so that every potential solution v to $D\mathrm{y} = \mathrm{b}$ can be decomposed into solutions $\mathrm{v}', \mathrm{v}''$ to the two new bricks. It is easy to see that this condition implies that in P, we may replace the brick $D\mathrm{y} = \mathrm{b}$ with $D\mathrm{y}' = \mathrm{b}'$ and $D\mathrm{y}'' = \mathrm{b}''$ without changing feasibility or, in the case of the optimization problem, the minimum value of the optimization goal. In the latter setting, both new bricks inherit the optimization vector c_i from the original brick.

Before we continue, let us comment on the proof of Lemma 8. We use two ingredients. The first one is the following fundamental result of Klein [29]. (Here, for a multiset of vectors A, by $\sum A$ we denote the sum of all the vectors in A.)

Lemma 9 (Klein Lemma, variant from [13]). Let $T_1, \ldots, T_n$ be non-empty multisets of vectors in $\mathbb{Z}^k$ such that $\sum T_1 = \sum T_2 = \ldots = \sum T_n$ and all vectors contained in all multisets $T_1, \ldots, T_n$ have ℓ_∞-norm bounded by Δ. Then there are non-empty multisets $S_1 \subseteq T_1, \ldots, S_n \subseteq T_n$, each of size at most $2^{\mathcal{O}(k\Delta)^k}$, such that $\sum S_1 = \sum S_2 = \ldots = \sum S_n$.

In the context of the proof of Lemma 8, we apply Lemma 9 to the family of all multisets T that consist of columns of D and satisfy $\sum T = \mathrm{b}$. By encoding multiplicities, such multisets correspond to vectors $\mathrm{v} \in \mathbb{Z}^k_{\geqslant 0}$ satisfying $D\mathrm{v} = \mathrm{b}$. (We hide here some technicalities regarding the fact that this family is infinite.) By Lemma 9, from each such multiset T, we can extract a submultiset S of bounded size such that all the submultisets S sum up to the same vector b'. Denoting $\mathrm{b}'' = \mathrm{b} - \mathrm{b}'$, this means that every vector $\mathrm{v} \in \mathbb{Z}^k_{\geqslant 0}$ satisfying $D\mathrm{v} = \mathrm{b}$ can be decomposed as $\mathrm{v} = \mathrm{v}' + \mathrm{v}''$ with $\mathrm{v}', \mathrm{v}'' \in \mathbb{Z}^k_{\geqslant 0}$ so that $D\mathrm{v}' = \mathrm{b}'$ and $D\mathrm{v}'' = \mathrm{b}''$. Namely, v' corresponds to the vectors contained in S and v'' corresponds to the vectors contained in $T - S$, where T is the multiset corresponding to v.

There is an issue in the above reasoning: we do not obtain the property $\mathrm{b}', \mathrm{b}'' \sqsubseteq \mathrm{b}$, which will be important in later applications of Lemma 8. To

bridge this difficulty, we apply the argument above exhaustively to decompose b as $\mathrm{b}_1 + \ldots + \mathrm{b}_m$, for some integer m, so that every vector b_i has the ℓ_∞-norm bounded by $2^{\mathcal{O}(k\Delta)^k}$ and every vector $\mathrm{v} \in \mathbb{Z}^k_{\geqslant 0}$ satisfying $D\mathrm{v} = \mathrm{b}$ can be decomposed as $\mathrm{v} = \mathrm{v}_1 + \ldots + \mathrm{v}_m$ where $\mathrm{v}_i \in \mathbb{Z}^k_{\geqslant 0}$ satisfies $D\mathrm{v}_i = \mathrm{b}_i$. Then, we treat vectors $\mathrm{b}_1, \ldots, \mathrm{b}_m$ with the following lemma.

Lemma 10 (see Lemma 29). Let $\mathrm{u}_1, \ldots, \mathrm{u}_m$ be vectors in $\mathbb{Z}^k$ of ℓ_∞-norm bounded by Ξ, and let $\mathrm{b} = \sum_{i=1}^m \mathrm{u}_i$. Then the vectors $\mathrm{u}_1, \ldots, \mathrm{u}_m$ can be grouped into non-empty groups $U_1, \ldots, U_\ell$, each of size at most $\mathcal{O}(\Delta)^{2^{k-1}}$, so that $\sum U_i \sqsubseteq \mathrm{b}$ for all $i = 1, \ldots, \ell$.

More precisely, Lemma 10 allows us to group vectors $\mathrm{b}_1, \ldots, \mathrm{b}_m$ into groups of bounded size so that the sum within each group is sign-compatible with b. Assuming $\|\mathrm{b}\|_\infty$ is large enough, there will be at least two groups. Then, any non-trivial partition of the groups translates into a suitable decomposition $\mathrm{b} = \mathrm{b}' + \mathrm{b}''$ with $\mathrm{b}', \mathrm{b}'' \sqsubseteq \mathrm{b}$.

The proof of Lemma 10 is by induction on k and uses arguments similar to standard proofs of Steinitz Lemma. This concludes a sketch of the proof of Lemma 8.

Once Lemma 8 is established, it is natural to use it iteratively: break b into $\mathrm{b}', \mathrm{b}''$, then break b' into two even smaller vectors, and so on. By applying the argument exhaustively, eventually we obtain a collection of vectors $\mathrm{b}_1, \ldots, \mathrm{b}_m \sqsubseteq \mathrm{b}$ such that $\mathrm{b} = \mathrm{b}_1 + \ldots + \mathrm{b}_m$, $\|\mathrm{b}_i\|_\infty \leqslant 2^{(k\Delta)^{\mathcal{O}(k)}}$ for all $i \in \{1, \ldots, m\}$, and every $\mathrm{v} \in \mathbb{Z}^k_{\geqslant 0}$ satisfying $D\mathrm{v} = \mathrm{b}$ can be decomposed as $\mathrm{v} = \mathrm{v}_1 + \ldots + \mathrm{v}_m$ with $\mathrm{v}_i \in \mathbb{Z}^k_{\geqslant 0}$ and $D\mathrm{v}_i = \mathrm{b}_i$ for all $i \in \{1, \ldots, m\}$. We call such a collection a faithful decompostion of b of order $2^{(k\Delta)^{\mathcal{O}(k)}}$.

There is an important technical caveat here. Observe that the size m of a faithful decomposition of a right-hand side b can be as large as $\Omega(\|\mathrm{b}\|_1)$, which is exponential in the bitsize of the program P. So we cannot hope to compute a faithful decomposition explicitly within the target time complexity. However, observe that all vectors b_i in a faithful decomposition $\mathcal{B}$ are bounded in ℓ_∞-norm by $\Xi \coloneqq 2^{(k\Delta)^{\mathcal{O}(k)}}$, and there are only at most $(2\Xi + 1)^k$ different such vectors. Therefore, $\mathcal{B}$ can be encoded by storing, for each vector b' present in $\mathcal{B}$, the multiplicity of b' in $\mathcal{B}$. Thus, describing $\mathcal{B}$ takes $2^{(k\Delta)^{\mathcal{O}(k)}} \cdot \log \|\mathrm{b}\|_\infty$ bits.

With this encoding scheme in mind, we show that a faithful decomposition $\mathcal{B}$ of a given vector b of order at most Ξ can be computed in fixed-parameter time $f(\Delta, k) \cdot (\log \|\mathrm{b}\|_\infty)^{\mathcal{O}(1)}$, for a computable function f. For this, we show that one can extract parts of the decomposition in "larger chunks", at each step reducing the ℓ_1-norm of the decomposed vector by a constant fraction; this gives a total number of steps logarithmic in $\|\mathrm{b}\|_1$. In each step, to extract the next large chunk of the decomposition, we use the fixed-parameter algorithm for optimization problems definable in Presburger arithmetic, due to Koutecký and Talmon [39]. We remark that in our context, this tool could be also replaced by the fixed-parameter algorithm of Eisenbrand and Shmonin [18] for $\forall\exists$ integer programming.

Reduction to (mixed) integer programming with few variables. With faithful decompositions understood, we can compute, for every right-hand side b_i part of P, a faithful decomposition $\{\mathrm{b}_i^1, \ldots, \mathrm{b}_i^{m_i}\}$ of b_i. This allows us to construct an equivalent (in terms of feasibility and optimization) n-fold program P' by replacing each brick $D\mathrm{y}_i = \mathrm{b}_i$ with bricks $D\mathrm{y}_i^j = \mathrm{b}_i^j$ for $j \in \{1, \ldots, m_i\}$. Thus, the program P' has an exponential number of bricks, but can be computed and described concisely: all right-hand sides are bounded in the ℓ_∞-norm by at most Ξ, so for every potential right-hand side b, we just write the multiplicity in which b appears in P'. We remark that such high-multiplicity encoding of n-fold integer programs has already been studied by Knop et al. [37].

For convenience, let $\mathsf{RHS} \coloneqq \{-\Xi, \ldots, \Xi\}^k$ be the set of all possible right-hand sides, and for $\mathrm{b} \in \mathsf{RHS}$, by $\mathsf{count}[\mathrm{b}]$ we denote the multiplicity of b in P'.

It is now important to better understand the set of solutions to a single brick $D\mathrm{y} = \mathrm{b}$ present in P'. Here comes a key insight stemming from the theory of Graver bases: as (essentially) proved by Pottier [44], every solution $\mathrm{w} \in \mathbb{Z}^k_{\geqslant 0}$ to $D\mathrm{w} = \mathrm{b}$ can be decomposed as $\mathrm{w} = \widehat{\mathrm{w}} + \mathrm{g}_1 + \ldots + \mathrm{g}_\ell$, where

- $\widehat{\mathrm{w}} \in \mathbb{Z}^k_{\geqslant 0}$ is a base solution that also satisfies $D\widehat{\mathrm{w}} = \mathrm{b}$, but $\|\widehat{\mathrm{w}}\|_\infty$ is bounded by a function of Δ and $\|\mathrm{b}\|_\infty$, and
- $\mathrm{g}_1, \ldots, \mathrm{g}_\ell \in \mathbb{Z}^k_{\geqslant 0}$ are elements of the Graver basis of D.

Here, the Graver basis of D consists of all conformally-minimal non-zero vectors g satisfying $D\mathrm{g} = 0$. In particular, it is known that the Graver basis is always finite and consists of vectors of ℓ_∞ norm bounded by $(2k\Delta + 1)^k$ [19]. The decomposition explained above will be called a Graver decomposition of w.

For $\mathrm{b} \in \mathsf{RHS}$, let $\mathsf{Base}[\mathrm{b}]$ be the set of all possible base solutions $\widehat{\mathrm{w}}$ to $D\mathrm{y} = \mathrm{b}$. As $\|\mathrm{b}\|_\infty \leqslant \Xi$ and Ξ is bounded by a function of the parameters under consideration, it follows that $\mathsf{Base}[\mathrm{b}]$ consists only of vectors of bounded ℓ_∞-norms, and therefore it can be efficiently constructed.

Having this understanding, we can write an integer program M with few variables that is equivalent to P'. The variables are as follows:

- For every $\mathrm{b} \in \mathsf{RHS}$ and $\widehat{\mathrm{w}} \in \mathsf{Base}[\mathrm{b}]$, we introduce a variable $\zeta^{\mathrm{b}}_{\widehat{\mathrm{w}}} \in \mathbb{Z}_{\geqslant 0}$ that signifies how many times in total $\widehat{\mathrm{w}}$ is used in the Graver decompositions of solutions to individual bricks.
- For every nonnegative vector g in the Graver basis of D, we introduce a variable $\delta_{\mathrm{g}} \in \mathbb{Z}_{\geqslant 0}$ signifying how many times in total g appears in the Graver decompositions of solutions to individual bricks.

Note that since program P' is uniform, the guessed base solutions and elements of the Graver basis can be assigned to any brick with the same effect on the linking constraints of P'. Hence, it suffices to verify the cardinalities and the total effect on the linking constrains of P', yielding the following constraints of M:

- the translated linking constraints: $\sum_{\mathrm{b} \in \mathsf{RHS}} \sum_{\widehat{\mathrm{w}} \in \mathsf{Base}[\mathrm{b}]} \zeta^{\mathrm{b}}_{\widehat{\mathrm{w}}} \cdot C\widehat{\mathrm{w}} + \sum_{\mathrm{g} \in \mathsf{Graver}(D), \mathrm{g} \geqslant 0} \delta_{\mathrm{g}} \cdot C\mathrm{g} = \mathrm{a}$.
- for every $\mathrm{b} \in \mathsf{RHS}$, the cardinality constraint $\sum_{\widehat{\mathrm{w}} \in \mathsf{Base}[\mathrm{b}]} \zeta^{\mathrm{b}}_{\widehat{\mathrm{w}}} = \mathsf{count}[\mathrm{b}]$.

Noting that the number of variables of M is bounded in terms of the parameters, we may apply any fixed-parameter algorithm for integer programming param-

eterized by the number of variables, for instance that of Kannan [28], to solve M. This concludes the description of the algorithm for the feasibility problem.

In the case of the optimization problem, there is an issue that the optimization vectors c_i may differ between different bricks, and there may be as many as n different such vectors. While the Graver basis elements can be always greedily assigned to bricks in which their contribution to the optimization goal is the smallest, this is not so easy for the base solutions, as every brick may accommodate only one base solution. We may enrich M by suitable assignment variables $\omega^{\mathrm{b},i}_{\widehat{\mathrm{w}}}$ to express how many base solutions of each type are assigned to bricks with different optimization vectors; but this yields as many as $\Omega(n)$ additional variables. Fortunately, we observe that in the enriched program M, if one fixes any integral valuation of variables $\zeta^{\mathrm{b}}_{\widehat{\mathrm{w}}}$ and δ_{g}, the remaining problem on variables $\omega^{\mathrm{b},i}_{\widehat{\mathrm{w}}}$ corresponds to a flow problem, and hence its constraint matrix is totally unimodular. Thus, we may solve M as a mixed integer program where variables $\omega^{\mathrm{b},i}_{\widehat{\mathrm{w}}}$ are allowed to be fractional. The number of integral variables is bounded in terms of parameters, so we may apply the fixed-parameter algorithm for mixed integer programming of Lenstra [43].

3 Preliminaries

We write $\mathbb{Z}_{\geqslant 0}$ and $\mathbb{R}_{\geqslant 0}$ for the sets of nonnegative integers and nonnegative reals, respectively.

Linear algebra. In the technical part of this paper we choose to use a somewhat unorthodox notation for linear algebra, which we introduce now. The correspondence between this notation and the standard notation used in the previous sections is straightforward.

A tuple of variables is just a finite set of variable names. We use the convention that tuples of variables are denoted $\mathrm{x}, \mathrm{y}, \mathrm{z}$, etc., while individual variable names are x, y, z, etc. For a set A and a tuple of variables x, an x-vector over A is a function $\mathrm{u}\colon \mathrm{x} \to A$. The value of u on a variable $x \in \mathrm{x}$ will be denoted by $\mathrm{u}[x]$, and we call it the x-entry of u. Vectors will be typically denoted by $\mathrm{a}, \mathrm{b}, \mathrm{c}, \mathrm{u}, \mathrm{v}, \mathrm{w}$ and so on. We write A^{x} for the set of all x-vectors over A. We can also apply arithmetics to vectors coordinate-wise in the standard manner, whenever the set A is endowed with a structure of a ring. When $A = \mathbb{R}$, as usual we write $\|\mathrm{u}\|_\infty \coloneqq \max_{x\in\mathrm{x}} |\mathrm{u}[x]|$ and $\|\mathrm{u}\|_1 \coloneqq \sum_{x\in\mathrm{x}} |\mathrm{u}[x]|$.

For a ring R (typically $\mathbb{Z}$ or $\mathbb{R}$) and tuples of variables x and y, an $\mathrm{x}\times\mathrm{y}$-matrix is an x-vector of y-vectors, that is, an element of $(R^{\mathrm{y}})^{\mathrm{x}}$; we denote the latter set as $R^{\mathrm{x}\times\mathrm{y}}$ for convenience. The reader should think of the individual y-vectors comprised in a matrix as of the columns of the said matrix. We use capital letters A, B, C, etc. for matrices. For $x \in \mathrm{x}$ and $y \in \mathrm{y}$, we write $A[x, y] \coloneqq (A[x])[y]$, that is, $A[x, y]$ is the y-entry of the column of A corresponding to variable x. As for vectors, by $\|A\|_\infty \coloneqq \max_{x\in\mathrm{x}, y\in\mathrm{y}} |A[x, y]|$ we denote the maximum absolute value of an entry of A.

For a matrix $A \in R^{\mathrm{x}\times\mathrm{y}}$ and a vector $\mathrm{u} \in R^{\mathrm{x}}$, $A\mathrm{u}$ is the vector $\mathrm{v} \in R^{\mathrm{y}}$ satisfying

$$\mathrm{v}[y] = \sum_{x\in\mathrm{x}} A[x,y]\cdot \mathrm{u}[x] \qquad \text{for all } y \in \mathrm{y}.$$

For vectors $\mathrm{u},\mathrm{v} \in R^{\mathrm{x}}$, the inner product of u and v is

$$\langle \mathrm{u},\mathrm{v}\rangle \coloneqq \sum_{x\in\mathrm{x}} \mathrm{u}[x]\cdot\mathrm{v}[x].$$

All-zero and all-one vectors are denoted by 0 and 1, respectively, and their domains will be always clear from the context.

Conformal order. Let x be a tuple of variables. Vectors $\mathrm{u},\mathrm{v} \in \mathbb{Z}^{\mathrm{x}}$ are sign-compatible if $\mathrm{u}[x]\cdot\mathrm{v}[x] \geqslant 0$ for all $x \in \mathrm{x}$; that is, on every coordinate u and v must have the same sign, where 0 is assumed to be compatible with both signs. The conformal order $\sqsubseteq$ on vectors in $\mathbb{Z}^{\mathrm{x}}$ is defined as follows: For two vectors $\mathrm{u},\mathrm{v} \in \mathbb{Z}^{\mathrm{x}}$, we have $\mathrm{u} \sqsubseteq \mathrm{v}$ if u and v are sign-compatible and

$$|\mathrm{u}[x]| \leqslant |\mathrm{v}[x]| \qquad \text{for all } x \in \mathrm{x}.$$

Note that thus, $\sqsubseteq$ is a partial order on $\mathbb{Z}^{\mathrm{x}}$.

Collections. A collection over a set A is a function $C\colon I \to A$, where I is the index set of C. We often use notation $C = \langle c_i \colon i \in I\rangle$ to enumerate the elements of a collection with the elements of its index set, where formally c_i is the value of C on i. Note that syntactically, a collection with index set I is just an I-vector, but we will use collections and vectors in different ways. Typically, when speaking about collections we are not really interested in the index set, and the reader can always assume it to be a prefix of natural numbers. However, in notation it will be convenient to consider various index sets.

Standard set theory notation can be used in the context of collections in the natural manner, by applying it to the index sets. In particular, a subcollection of a collection $C = \langle c_i \colon i \in I\rangle$ is any collection $C' = \langle c_i \colon i \in I'\rangle$ for $I' \subseteq I$.

Block-structured integer programming. We now introduce the two variants of block-structured integer programming problems that we are interested in: two-stage stochastic integer programming and n-fold integer programming.

A two-stage stochastic integer program consists of three collections:

- $\langle A_i \colon i \in I\rangle$ is a collection of matrices in $\mathbb{Z}^{\mathrm{x}\times\mathrm{t}}$;
- $\langle D_i \colon i \in I\rangle$ is a collection of matrices in $\mathbb{Z}^{\mathrm{y}\times\mathrm{t}}$; and
- $\langle \mathrm{b}_i \colon i \in I\rangle$ is a collection of vectors in $\mathbb{Z}^{\mathrm{t}}$.

Here, $\mathrm{x},\mathrm{y},\mathrm{t}$ are tuples of variables and I is the index set of the program. A solution to such a program consists of a vector $\mathrm{u} \in \mathbb{Z}^{\mathrm{x}}_{\geqslant 0}$ and vectors $\langle \mathrm{v}_i \colon i \in I\rangle$ in $\mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ such that

$$A_i\mathrm{u} + D_i\mathrm{v}_i = \mathrm{b}_i \quad \text{for all } i \in I.$$

In the Two-Stage Stochastic ILP Feasability problem, we are given a two-stage stochastic integer program in the form described above, and the task is to decide whether this program has a solution.

An n-fold integer program consists of the following components:

- $\langle C_i \colon i \in I \rangle$ is a collection of matrices in $\mathbb{Z}^{\mathrm{y} \times \mathrm{s}}$;
- $\langle D_i \colon i \in I \rangle$ is a collection of matrices in $\mathbb{Z}^{\mathrm{y} \times \mathrm{t}}$;
- a is a vector in $\mathbb{Z}^{\mathrm{s}}$; and
- $\langle \mathrm{b}_i \colon i \in I \rangle$ is a collection of vectors in $\mathbb{Z}^{\mathrm{t}}$.

Again, $\mathrm{y}, \mathrm{s}, \mathrm{t}$ are tuples of variables and I is the index set of the program. A uniform program is one where $C_i = C_j$ for all $i, j \in I$. A solution to the program described above consists of vectors $\langle \mathrm{v}_i \colon i \in I \rangle$ in $\mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ such that

$$\sum_{i \in I} C_i \mathrm{v}_i = \mathrm{a} \qquad \text{and} \qquad D_i \mathrm{v}_i = \mathrm{b}_i \quad \text{for all } i \in I.$$

The n-fold ILP Feasability problem asks for the existence of a solution to a given n-fold integer program, while in the n-fold ILP Optimization problem, the task is to minimize $\sum_{i \in I} \langle \mathrm{c}_i, \mathrm{v}_i \rangle$ among the solutions, for additionally given vectors $\langle \mathrm{c}_i \colon i \in I \rangle$ in $\mathbb{Z}^{\mathrm{y}}$. A prefix Uniform may be added to the problem name to specify that we speak about uniform programs. (Note that only the matrices C_i have to be equal, and not necessarily the optimization goal vectors c_i.)

For an integer program P, be it a two-stage stochastic or an n-fold program, by $\|P\|$ we denote the total bitsize of the encoding of P, where all numbers are encoded in binary.

4 Two-stage stochastic integer programming

Our main result for two-stage stochastic integer programs is captured in the following statement, which is Theorem 1 with adjusted notation.

Theorem 11. An instance $P = \langle A_i, D_i, \mathrm{b}_i \colon i \in I \rangle$ of Two-Stage Stochastic ILP Feasability can be solved in time $f(\Delta, |\mathrm{x}|, |\mathrm{t}|) \cdot \|P\|$ for some computable function f, where $\Delta = \max_{i \in I} \|D_i\|_\infty$.

Note that importantly, the absolute values of the entries of matrices A_i are not assumed to be bounded in terms of the considered parameters. Also, in Theorem 11 the parameters do not include $|\mathrm{y}|$, the number of columns of each block D_i. This is because by removing equal columns in those blocks (which does not affect the feasibility of the program), we may always assume $|\mathrm{y}| \leqslant (2\Delta+1)^{|\mathrm{t}|}$.

The proof of Theorem 11 spans the entirety of this section.

4.1 Cones

In our proof we will rely on the geometry of polyhedral and integer cones. For this, we introduce the following notation. Let t be a tuple of variables and

$\mathcal{D} \subseteq \mathbb{Z}^t$ be a finite set of vectors. For $S \in \{\mathbb{R}, \mathbb{R}_{\geqslant 0}, \mathbb{Z}, \mathbb{Z}_{\geqslant 0}\}$, we define the S-span of $\mathcal{D}$ as

$$\mathsf{span}^S(\mathcal{D}) \coloneqq \left\{ \sum_{d \in \mathcal{D}} \lambda_d \cdot d \ : \ \langle \lambda_d \colon d \in \mathcal{D} \rangle \text{ is a collection of elements of } S \right\} \subseteq \mathbb{R}^t.$$

In other words, $\mathsf{span}^S(\mathcal{D})$ are all vectors that can be expressed as a linear combination of vectors in $\mathcal{D}$ with coefficients belonging to S. For clarity, we write

$$\mathsf{span}(\mathcal{D}) \coloneqq \mathsf{span}^{\mathbb{R}}(\mathcal{D}), \qquad \mathsf{cone}(\mathcal{D}) \coloneqq \mathsf{span}^{\mathbb{R}_{\geqslant 0}}(\mathcal{D}),$$
$$\mathsf{lattice}(\mathcal{D}) \coloneqq \mathsf{span}^{\mathbb{Z}}(\mathcal{D}), \qquad \mathsf{intCone}(\mathcal{D}) \coloneqq \mathsf{span}^{\mathbb{Z}_{\geqslant 0}}(\mathcal{D}).$$

Thus, $\mathsf{span}(\mathcal{D})$ is just the usual linear space spanned of $\mathcal{D}$. Next, $\mathsf{cone}(\mathcal{D})$ is the standard polyhedral cone: it consists of all vectors expressible as nonnegative linear combinations of vectors in $\mathcal{D}$. Similarly, $\mathsf{intCone}(\mathcal{D})$ is the integer cone, where we restrict attention to nonnegative integer combinations of vectors in $\mathcal{D}$. Finally, $\mathsf{lattice}(\mathcal{D})$ is the integer lattice consisting of all vectors that can be reached from 0 by adding and subtracting vectors of $\mathcal{D}$.

We remark that while the reader may think of $\mathcal{D}$ as of a matrix whose columns are the vectors of $\mathcal{D}$, ordered arbitrarily, in the notation we will consistently treat $\mathcal{D}$ as a set of vectors. Consequently, $|\mathcal{D}|$ denotes the number of vectors in $\mathcal{D}$ (or, the number of columns of $\mathcal{D}$ treated as a matrix), and the same also applies to all subsets of $\mathcal{D}$.

4.1.1 Dual representation of cones

We will use the classic result of Weyl about representing polyhedral cones as intersections of half-spaces.

Theorem 12 (Weyl, [48]). For every set of vectors $\mathcal{D} \subseteq \mathbb{Z}^t$ there exists a finite set of vectors $\mathcal{F} \subseteq \mathbb{Z}^t$ such that

$$\mathsf{cone}(\mathcal{D}) = \{v \in \mathbb{R}^t \mid \langle f, v \rangle \geqslant 0 \text{ for all } f \in \mathcal{F}\}.$$

Moreover, such a set $\mathcal{F}$ can be computed given $\mathcal{D}$.

A set $\mathcal{F}$ satisfying the outcome of Theorem 12 will be called a dual representation of $\mathsf{cone}(\mathcal{D})$. Here is a simple observation about how elements of the dual representation relate to the elements of $\mathcal{D}$.

Observation 13. Suppose $\mathcal{F}$ is a dual representation of $\mathsf{cone}(\mathcal{D})$ for a set of vectors $\mathcal{D} \subseteq \mathbb{Z}^t$. Then

$$\langle f, d \rangle \geqslant 0 \quad \text{for all } \ f \in \mathcal{F} \text{ and } \ d \in \mathcal{D}.$$

Proof. Clearly $d \in \mathsf{cone}(\mathcal{D})$, so $\langle f, d \rangle \geqslant 0$ because $\mathcal{F}$ is a dual representation of $\mathsf{cone}(\mathcal{D})$. □

For a subset $\mathcal{G} \subseteq \mathcal{F}$, we define

$$\mathcal{D}_{\mathcal{G}} \coloneqq \{\mathrm{d} \in \mathcal{D} \mid \langle \mathrm{g}, \mathrm{d} \rangle = 0 \text{ for all } \mathrm{g} \in \mathcal{G}\}.$$

In other words, $\mathcal{D}_{\mathcal{G}}$ consists of those vectors of $\mathcal{D}$ that are orthogonal to all vectors in $\mathcal{G}$. The next lemma expresses the following intuition: the facet of $\mathsf{cone}(\mathcal{D})$ corresponding to $\mathcal{G}$ is spanned by the vectors of $\mathcal{D}_{\mathcal{G}}$.

Lemma 14. Suppose $\mathcal{F}$ is a dual representation of $\mathsf{cone}(\mathcal{D})$ for a set of vectors $\mathcal{D} \subseteq \mathbb{Z}^{\mathrm{t}}$. Then for every subset $\mathcal{G} \subseteq \mathcal{F}$ we have

$$\mathsf{cone}(\mathcal{D}_{\mathcal{G}}) = \mathsf{cone}(\mathcal{D}) \cap \{\mathrm{v} \in \mathbb{R}^{\mathrm{t}} \mid \langle \mathrm{g}, \mathrm{v} \rangle = 0 \text{ for all } \mathrm{g} \in \mathcal{G}\}.$$

Proof. Note that $\mathcal{D}_{\mathcal{G}} \subseteq \mathcal{D}$ entails $\mathsf{cone}(\mathcal{D}_{\mathcal{G}}) \subseteq \mathsf{cone}(\mathcal{D})$. Further, since $\langle \mathrm{g}, \mathrm{d} \rangle = 0$ for all $\mathrm{g} \in \mathcal{G}$ and all $\mathrm{d} \in \mathcal{D}_{\mathcal{G}}$, the same also holds for all linear combinations of vectors in $\mathcal{D}_{\mathcal{G}}$, implying that $\langle \mathrm{g}, \mathrm{v} \rangle = 0$ for all $\mathrm{g} \in \mathcal{G}$ and $\mathrm{v} \in \mathsf{cone}(\mathcal{D}_{\mathcal{G}})$. This proves inclusion $\subseteq$.

For the converse inclusion, consider any $\mathrm{v} \in \mathsf{cone}(\mathcal{D})$ such that $\langle \mathrm{g}, \mathrm{v} \rangle = 0$ for all $\mathrm{g} \in \mathcal{G}$. As $\mathrm{v} \in \mathsf{cone}(\mathcal{D})$, there is a collection of nonnegative reals $\langle \lambda_{\mathrm{d}} \colon \mathrm{d} \in \mathcal{D} \rangle$ such that

$$\mathrm{v} = \sum_{\mathrm{d} \in \mathcal{D}} \lambda_{\mathrm{d}} \cdot \mathrm{d}.$$

Consider any $\mathrm{e} \in \mathcal{D} - \mathcal{D}_{\mathcal{G}}$. By the definition of $\mathcal{D}_{\mathcal{G}}$ and Observation 13, there exists $\mathrm{g} \in \mathcal{G}$ such that $\langle \mathrm{g}, \mathrm{e} \rangle > 0$. As all coefficients $\langle \lambda_{\mathrm{d}} \colon \mathrm{d} \in \mathcal{D} \rangle$ are nonnegative, by Observation 13 we have

$$0 = \langle \mathrm{g}, \mathrm{v} \rangle = \sum_{\mathrm{d} \in \mathcal{D}} \lambda_{\mathrm{d}} \cdot \langle \mathrm{g}, \mathrm{d} \rangle \geqslant \lambda_{\mathrm{e}} \cdot \langle \mathrm{g}, \mathrm{e} \rangle.$$

As $\langle \mathrm{g}, \mathrm{e} \rangle > 0$, we necessarily have $\lambda_{\mathrm{e}} = 0$; and this holds for every $\mathrm{e} \in \mathcal{D} - \mathcal{D}_{\mathcal{G}}$. So in fact v is a nonnegative linear combination of vectors in $\mathcal{D}_{\mathcal{G}}$, or equivalently $\mathrm{v} \in \mathsf{cone}(\mathcal{D}_{\mathcal{G}})$. This proves inclusion $\supseteq$. □

4.1.2 Membership in integer cones and in integer lattices

First, we need a statement that membership in an integer lattice can be witnessed by a vector with small entries. The following lemma follows from standard bounds given by Pottier [44, Corollary 4] (c.f. Lemma 26).

Lemma 15. For every set of vectors $\mathcal{D} \subseteq \mathbb{Z}^{\mathrm{t}}$ and a vector $\mathrm{v} \in \mathsf{lattice}(\mathcal{D})$, there exist integer coefficients $\langle \lambda_{\mathrm{d}} \colon \mathrm{d} \in \mathcal{D} \rangle$ such that

$$\mathrm{v} = \sum_{\mathrm{d} \in \mathcal{D}} \lambda_{\mathrm{d}} \cdot \mathrm{d} \qquad \text{and} \qquad \sum_{\mathrm{d} \in \mathcal{D}} |\lambda_{\mathrm{d}}| \leqslant \left(2 + \max_{\mathrm{d} \in \mathcal{D}} \|\mathrm{d}\|_\infty + \|\mathrm{v}\|_\infty\right)^{2|\mathrm{t}|}.$$

We now proceed to a technical statement that is crucial in our approach. Intuitively, it says that if a vector lies "deep" in the polyhedral cone, then its membership in the integer cone is equivalent to the membership in the integer lattice.

Lemma 16 (Deep-in-the-Cone Lemma). Let $\mathcal{D} \subseteq \mathbb{Z}^t$ be a set of vectors and $\mathcal{F} \subseteq \mathbb{Z}^t$ be a dual representation of $\mathsf{cone}(\mathcal{D})$. Then there is a positive integer M, computable from $\mathcal{D}$ and $\mathcal{F}$, such that for every $\mathcal{G} \subseteq \mathcal{F}$ and every $v \in \mathsf{cone}(\mathcal{D}_\mathcal{G})$ satisfying $\langle f, v\rangle \geqslant M$ for all $f \in \mathcal{F} - \mathcal{G}$, we have

$$v \in \mathsf{lattice}(\mathcal{D}_\mathcal{G}) \quad \text{if and only if} \quad v \in \mathsf{intCone}(\mathcal{D}_\mathcal{G}).$$

Proof. We set

$$M \coloneqq L \cdot |\mathcal{D}| \cdot \max_{f \in \mathcal{F}} \|f\|_1 \cdot \max_{d \in \mathcal{D}} \|d\|_\infty, \qquad \text{where} \qquad L \coloneqq \left(2 + (|\mathcal{D}| + 1) \cdot \max_{d \in \mathcal{D}} \|d\|_\infty\right)^{2|t|}$$

is the bound provided by Lemma 15 for vectors of ℓ_∞-norm bounded by $|\mathcal{D}| \cdot \max_{d \in \mathcal{D}} \|d\|_\infty$.

As $\mathsf{intCone}(\mathcal{D}_\mathcal{G}) \subseteq \mathsf{lattice}(\mathcal{D}_\mathcal{G})$, it suffices to prove the following: if $v \in \mathsf{cone}(\mathcal{D}_\mathcal{G}) \cap \mathsf{lattice}(\mathcal{D}_\mathcal{G})$ satisfies $\langle f, v\rangle \geqslant M$ for all $f \in \mathcal{F} - \mathcal{G}$, then in fact $v \in \mathsf{intCone}(\mathcal{D}_\mathcal{G})$. Let

$$w \coloneqq \sum_{d \in \mathcal{D}_\mathcal{G}} L \cdot d.$$

As both v and w are linear combinations of vectors in $\mathcal{D}_\mathcal{G}$, we have

$$\langle g, v - w\rangle = 0 \qquad \text{for all } g \in \mathcal{G}.$$

Further, for each $f \in \mathcal{F} - \mathcal{G}$ we have

$$\langle f, v - w\rangle = \langle f, v\rangle - \langle f, w\rangle \geqslant M - L \cdot \sum_{d \in \mathcal{D}} |\langle f, d\rangle| \geqslant M - L \cdot |\mathcal{D}| \cdot \max_{f \in \mathcal{F}} \|f\|_1 \cdot \max_{d \in \mathcal{D}} \|d\|_\infty = 0.$$

As $\mathcal{F}$ is a dual representation of $\mathsf{cone}(\mathcal{D})$, the two assertions above together with Lemma 14 imply that $v - w \in \mathsf{cone}(\mathcal{D}_\mathcal{G})$. Consequently, there is a collection of nonnegative reals $\langle \lambda_d \colon d \in \mathcal{D}_\mathcal{G}\rangle$ such that

$$v - w = \sum_{d \in \mathcal{D}_\mathcal{G}} \lambda_d \cdot d.$$

Now, consider the vector

$$v' \coloneqq w + \sum_{d \in \mathcal{D}_\mathcal{G}} \lfloor \lambda_d \rfloor \cdot d = \sum_{d \in \mathcal{D}_\mathcal{G}} (L + \lfloor \lambda_d \rfloor) \cdot d.$$

Clearly $v' \in \mathsf{lattice}(\mathcal{D}_\mathcal{G})$. As $v \in \mathsf{lattice}(\mathcal{D}_\mathcal{G})$ by assumption, we have $v - v' \in \mathsf{lattice}(\mathcal{D}_\mathcal{G})$ as well. Furthermore,

$$\|v - v'\|_\infty = \left\| \sum_{d \in \mathcal{D}_\mathcal{G}} (\lambda_d - \lfloor \lambda_d \rfloor) \cdot d \right\|_\infty \leqslant |\mathcal{D}_\mathcal{G}| \cdot \max_{d \in \mathcal{D}_\mathcal{G}} \|d\|_\infty \leqslant |\mathcal{D}| \cdot \max_{d \in \mathcal{D}} \|d\|_\infty.$$

By Lemma 15 we conclude that there exist integer coefficients $\langle \mu_d \colon d \in \mathcal{D}_\mathcal{G}\rangle$ such that

$$v - v' = \sum_{d \in \mathcal{D}_\mathcal{G}} \mu_d \cdot d \qquad \text{and} \qquad \sum_{d \in \mathcal{D}_\mathcal{G}} |\mu_d| \leqslant L.$$

Hence,

$$\mathrm{v} = \mathrm{v}' + (\mathrm{v} - \mathrm{v}') = \sum_{\mathrm{d} \in \mathcal{D}_\mathcal{G}} (L + \lfloor \lambda_\mathrm{d} \rfloor + \mu_\mathrm{d}) \cdot \mathrm{d}.$$

It now remains to observe that each coefficient $L + \lfloor \lambda_\mathrm{d} \rfloor + \mu_\mathrm{d}$ is a nonnegative integer, because $\lambda_\mathrm{d} \geqslant 0$ and $|\mu_\mathrm{d}| \leqslant L$. This shows that $\mathrm{v} \in \mathsf{intCone}(\mathcal{D}_\mathcal{G})$, thereby completing the proof. □

4.2 Regular lattices

As mentioned in Section 1, the key idea in our approach is to guess the remainders of the entries of the sought solution modulo some large integer. To describe this idea formally, we consider regular lattices defined as follows.

Let K be a positive integer. For a vector of residues $\mathrm{r} \in \{0, 1, \ldots, K-1\}^\mathrm{t}$, where t is a tuple of variables, we define the regular lattice Λ_r^K:

$$\Lambda_\mathrm{r}^K \coloneqq \{\mathrm{v} \in \mathbb{Z}^\mathrm{t} \mid \mathrm{v}(t) \equiv \mathrm{r}(t) \bmod K \text{ for all } t \in \mathrm{t}\}.$$

In other words, Λ_r^K comprises all integer vectors in which the remainders mod K on all coordinates are exactly as specified in r. Note that regular lattices are affine rather than linear. In particular, $0 \in \Lambda_\mathrm{r}^K$ if and only if $\mathrm{r} = 0$, and hence a regular lattice Λ_r^K is not the form $\mathsf{lattice}(\mathcal{D})$ for a multiset of vectors $\mathcal{D}$, unless $\mathrm{r} = 0$.

We will need the following simple claim about fractionality of solutions to systems of equations.

Lemma 17. Let $\mathcal{D} \subseteq \mathbb{Z}^\mathrm{t}$ be a set of vectors, where t is a tuple of variables. Then there is a positive integer C, computable from $\mathcal{D}$, satisfying the following: for every $\mathrm{v} \in \mathsf{span}(\mathcal{D}) \cap \mathbb{Z}^\mathrm{t}$ there exist coefficients $\langle \lambda_\mathrm{d} \colon \mathrm{d} \in \mathcal{D} \rangle$ such that

$$\mathrm{v} = \sum_{\mathrm{d} \in \mathcal{D}} \lambda_\mathrm{d} \cdot \mathrm{d} \qquad \text{and} \qquad C\lambda_\mathrm{d} \text{ is an integer for every } \mathrm{d} \in \mathcal{D}.$$

Proof. Let z be a tuple of variables containing one variable z_d for each $\mathrm{d} \in \mathcal{D}$, and let D be the $\mathrm{z} \times \mathrm{t}$ matrix such that the column of D corresponding to $z_\mathrm{d} \in \mathrm{z}$ is d. Thus, collections of coefficients $\langle \lambda_\mathrm{d} \colon \mathrm{d} \in \mathcal{D} \rangle$ as in the lemma statement correspond to solutions $\boldsymbol{\lambda} \in \mathbb{Z}^\mathrm{z}$ of the system of equations $D\mathrm{z} = \mathrm{v}$. By first restricting the rows of D to a linearly independent set of rows, and then adding some $\{0,1\}$ row vectors together with zeroes on the right hand side, we can obtain a system of equations $\widetilde{D}\mathrm{z} = \widetilde{\mathrm{v}}$ such that every solution to $\widetilde{D}\mathrm{z} = \widetilde{\mathrm{v}}$ is also a solution to $D\mathrm{z} = \mathrm{v}$, and $\widetilde{D}$ is a non-singular square matrix. It now follows from Cramer rules that if $\boldsymbol{\lambda} \in \mathbb{Z}^\mathrm{z}$ is the unique solution to $\widetilde{D}\mathrm{z} = \widetilde{\mathrm{v}}$, then $\det \widetilde{D} \cdot \boldsymbol{\lambda}$ is an integer. Therefore, we may set $C \coloneqq \det \widetilde{D}$, where $\widetilde{D}$ is any non-singular square matrix that can be obtained from D by first restricting it to a linearly independent set of rows, and then extending this set of rows to a row basis using $\{0,1\}$ row vectors. Note that C is clearly computable from D. □

We now use Lemma 17 to prove the following statement that relates regular lattices with integer lattices generated by sets of vectors.

Lemma 18. Let $\mathcal{D} \subseteq \mathbb{Z}^{\mathrm{t}}$ be a set of vectors, where t is a tuple of variables. Then there is a positive integer K, computable from $\mathcal{D}$, satisfying the following: for every positive integer K' divisible by K and every vector of residues $\mathrm{r} \in \{0, 1, \ldots, K'-1\}^{\mathrm{t}}$,

$$\mathsf{span}(\mathcal{D}) \cap \Lambda_{\mathrm{r}}^{K'} \subseteq \mathsf{lattice}(\mathcal{D}) \qquad \text{or} \qquad \mathsf{lattice}(\mathcal{D}) \cap \Lambda_{\mathrm{r}}^{K'} = \emptyset.$$

Proof. We set $K \coloneqq C$, where C is the integer provided by Lemma 17 for $\mathcal{D}$. Fix any positive integer K' divisible by K and $\mathrm{r} \in \{0, 1, \ldots, K'-1\}^{\mathrm{t}}$. It suffices to prove that if there exists $\mathrm{u} \in \mathsf{lattice}(\mathcal{D}) \cap \Lambda_{\mathrm{r}}^{K'}$, then in fact $\mathsf{span}(\mathcal{D}) \cap \Lambda_{\mathrm{r}}^{K'} \subseteq \mathsf{lattice}(\mathcal{D})$. Consider any $\mathrm{v} \in \mathsf{span}(\mathcal{D}) \cap \Lambda_{\mathrm{r}}^{K'}$. Since $\mathrm{u}, \mathrm{v} \in \Lambda_{\mathrm{r}}^{K'}$, every entry of the vector $\mathrm{u} - \mathrm{v}$ is divisible by K', so also by K. It follows that

$$\mathrm{u} - \mathrm{v} = K \cdot \mathrm{w} \quad \text{for some } \mathrm{w} \in \mathbb{Z}^{\mathrm{t}}.$$

Since $\mathrm{u}, \mathrm{v} \in \mathsf{span}(\mathcal{D})$, we also have $\mathrm{w} \in \mathsf{span}(\mathcal{D})$. By Lemma 17, there exist coefficients $\langle \lambda_{\mathrm{d}} \colon \mathrm{d} \in \mathcal{D} \rangle$ such that

$$\mathrm{w} = \sum_{\mathrm{d} \in \mathcal{D}} \lambda_{\mathrm{d}} \cdot \mathrm{d} \qquad \text{and} \qquad K\lambda_{\mathrm{d}} \text{ is an integer for all } \mathrm{d} \in \mathcal{D}.$$

Therefore, we have

$$\mathrm{u} - \mathrm{v} = K \cdot \mathrm{w} = \sum_{\mathrm{d} \in \mathcal{D}} (K\lambda_{\mathrm{d}}) \cdot \mathrm{d},$$

where coefficients $K\lambda_{\mathrm{d}}$ are integral. So $\mathrm{u} - \mathrm{v} \in \mathsf{lattice}(\mathcal{D})$. As $\mathrm{u} \in \mathsf{lattice}(\mathcal{D})$ by assumption, we conclude that $\mathrm{v} \in \mathsf{lattice}(\mathcal{D})$ as well; this concludes the proof. □

4.3 Reduction to polyhedral constraints

We proceed to the cornerstone of our approach: the theorem stated below, which was presented in Section 2 as Theorem 4. Intuitively, it says that under fixing residues modulo a large integer, membership in an integer cone is equivalent to membership in a carefully crafted polyhedron.

Theorem 19 (Reduction to Polyhedral Constraints). Let $\mathcal{D} \subseteq \mathbb{Z}^{\mathrm{t}}$ be a set of vectors, where t is a tuple of variables. Then there exists a positive integer B, computable from $\mathcal{D}$, satisfying the following: for every vector of residues $\mathrm{r} \in \{0, 1, \ldots, B-1\}^{\mathrm{t}}$, there is a finite set $\mathcal{Q} \subseteq \mathbb{Z}^{\mathrm{t}} \times \mathbb{Z}$ such that

$$\Lambda_{\mathrm{r}}^{B} \cap \mathsf{intCone}(\mathcal{D}) = \Lambda_{\mathrm{r}}^{B} \cap \left\{ \mathrm{v} \in \mathbb{R}^{\mathrm{t}} \mid \langle \mathrm{q}, \mathrm{v} \rangle \geqslant a \text{ for all } (\mathrm{q}, a) \in \mathcal{Q} \right\}.$$

Moreover, there is an algorithm that given $\mathcal{D}$ and r, computes $\mathcal{Q}$ satisfying the above.

Proof. Let $\mathcal{F}$ be a dual representation of $\mathsf{cone}(\mathcal{D})$, computed from $\mathcal{D}$ using Theorem 12. Further, let

- K be the least common multiple of all integers $K_{\mathcal{G}}$ obtained by applying Lemma 18 to $\mathcal{D}_{\mathcal{G}}$ for every $\mathcal{G} \subseteq \mathcal{F}$, and
- M be the integer obtained by applying Lemma 16 to $\mathcal{D}$ and $\mathcal{F}$.

We define

$$\widehat{M} \coloneqq M + |\mathcal{D}| \cdot \max_{\mathrm{f} \in \mathcal{F}} \|\mathrm{f}\|_1 \cdot \max_{\mathrm{d} \in \mathcal{D}} \|\mathrm{d}\|_\infty,$$

and set

$$B_0 \coloneqq K, \qquad B_i \coloneqq \widehat{M} \cdot B_{i-1} \quad \text{for } i = 1, 2, \ldots, |\mathcal{F}|, \qquad \text{and} \qquad B \coloneqq 2 \cdot B_{|\mathcal{F}|}.$$

We are left with constructing a suitable set $\mathcal{Q}$ for a given $\mathrm{r} \in \{0, 1, \ldots, B-1\}^{\mathrm{t}}$.

For convenience, denote

$$\mathcal{R} \coloneqq \Lambda_{\mathrm{r}}^B \cap \mathsf{cone}(\mathcal{D}) \qquad \text{and} \qquad \mathcal{Z} \coloneqq \Lambda_{\mathrm{r}}^B \cap \mathsf{intCone}(\mathcal{D}).$$

For every $\mathrm{f} \in \mathcal{F}$, let p_{f} be the unique integer in $\{0, 1, \ldots, B-1\}$ such that

$$p_{\mathrm{f}} \equiv \langle \mathrm{f}, \mathrm{r} \rangle \mod B.$$

Observe the following.

Claim 20. For every $\mathrm{f} \in \mathcal{F}$ and $\mathrm{v} \in \mathcal{R}$, we have

$$\langle \mathrm{f}, \mathrm{v} \rangle \in \{p_{\mathrm{f}}, p_{\mathrm{f}} + B, p_{\mathrm{f}} + 2B, \ldots\}.$$

Proof. As $\mathrm{v} \in \mathsf{cone}(\mathcal{D})$ and $\mathcal{F}$ is a dual representation of $\mathsf{cone}(\mathcal{D})$, we have $\langle \mathrm{f}, \mathrm{v} \rangle \geqslant 0$. As $\mathrm{v} \in \Lambda_{\mathrm{r}}^B$, we also have $\langle \mathrm{f}, \mathrm{v} \rangle \equiv \langle \mathrm{f}, \mathrm{r} \rangle \equiv p_{\mathrm{f}} \bmod B$. The claim follows. □

For a given vector $\mathrm{v} \in \mathcal{R}$, call $\mathrm{f} \in \mathcal{F}$ tight for v if $\langle \mathrm{v}, \mathrm{f} \rangle = p_{\mathrm{f}}$. Importantly, Claim 20 implies the following: if f is not tight for v, then $\langle \mathrm{f}, \mathrm{v} \rangle \geqslant p_{\mathrm{f}} + B$. Further, let

$$\mathsf{Tight}(\mathrm{v}) \coloneqq \{\mathrm{f} \in \mathcal{F} \mid \mathrm{f} \text{ is tight for } \mathrm{v}\}.$$

The next claim is the key step. Its proof relies on a carefully crafted application of the Deep-in-the-Cone Lemma (Lemma 16).

Claim 21. Suppose $\mathrm{u} \in \mathcal{Z}$ and $\mathrm{v} \in \mathcal{R}$ are such that $\mathsf{Tight}(\mathrm{u}) \supseteq \mathsf{Tight}(\mathrm{v})$. Then $\mathrm{v} \in \mathcal{Z}$ as well.

Proof. Denote $\ell \coloneqq |\mathcal{F}|$ for brevity. Enumerate $\mathcal{F}$ as $\{\mathrm{f}_1, \mathrm{f}_2, \ldots, \mathrm{f}_\ell\}$ so that

$$\langle \mathrm{f}_1, \mathrm{v} \rangle \leqslant \langle \mathrm{f}_2, \mathrm{v} \rangle \leqslant \langle \mathrm{f}_3, \mathrm{v} \rangle \leqslant \ldots \leqslant \langle \mathrm{f}_\ell, \mathrm{v} \rangle,$$

and recall that all these values are nonnegative integers due to v being an integer vector in $\mathsf{cone}(\mathcal{D})$ and $\mathcal{F}$ being the dual representation of $\mathsf{cone}(\mathcal{D})$. Let $k \in \{0, 1, \ldots, \ell\}$ be the largest index such that

$$\langle \mathrm{f}_i, \mathrm{v} \rangle \leqslant B_i \qquad \text{for all } i \in \{1, 2, \ldots, k\},$$

and denote

$$\mathcal{G} \coloneqq \{\mathrm{f}_1, \mathrm{f}_2, \ldots, \mathrm{f}_k\} \qquad \text{and} \qquad \widehat{B} \coloneqq B_k.$$

By the maximality of k it follows that either $k = \ell$, or $k < \ell$ and $\langle \mathrm{f}_{k+1}, \mathrm{v}\rangle > B_{k+1}$. In any case, by the choice of the sequence $B_0, B_1, \ldots, B_\ell$ we have

$$\langle \mathrm{g}, \mathrm{v}\rangle \leqslant \widehat{B} \quad \text{for all } \mathrm{g} \in \mathcal{G} \qquad \text{and} \qquad \langle \mathrm{f}, \mathrm{v}\rangle > \widehat{M}\widehat{B} \quad \text{for all } \mathrm{f} \in \mathcal{F} - \mathcal{G}. \tag{3}$$

Since $\widehat{B} < B$, all vectors of $\mathcal{G}$ are tight for v; see Claim 20. As $\mathsf{Tight}(\mathrm{u}) \supseteq \mathsf{Tight}(\mathrm{v})$, these vectors are tight for u as well. We conclude that

$$\langle \mathrm{g}, \mathrm{u}\rangle = \langle \mathrm{g}, \mathrm{v}\rangle = p_{\mathrm{g}} \qquad \text{for all } \mathrm{g} \in \mathcal{G}. \tag{4}$$

Since $\mathrm{u} \in \mathcal{Z}$ by assumption, there is a collection of nonnegative integers $\langle \lambda_{\mathrm{d}} \colon \mathrm{d} \in \mathcal{D}\rangle$ such that

$$\mathrm{u} = \sum_{\mathrm{d}\in\mathcal{D}} \lambda_{\mathrm{d}} \cdot \mathrm{d}.$$

Define now a collection of nonnegative integers $\langle \mu_{\mathrm{d}} \colon \mathrm{d} \in \mathcal{D}\rangle$ as follows:

- if $\mathrm{d} \notin \mathcal{D}_{\mathcal{G}}$, then $\mu_{\mathrm{d}} = \lambda_{\mathrm{d}}$; and
- otherwise, if $\mathrm{d} \in \mathcal{D}_{\mathcal{G}}$, then μ_{d} is the unique integer in $\{0, 1, \ldots, K-1\}$ such that $\lambda_{\mathrm{d}} \equiv \mu_{\mathrm{d}} \bmod K$.

Consider now any $\mathrm{e} \in \mathcal{D} - \mathcal{D}_{\mathcal{G}}$. By the definition of $\mathcal{D}_{\mathcal{G}}$, there exists $\mathrm{g} \in \mathcal{G}$ such that $\langle \mathrm{g}, \mathrm{e}\rangle > 0$, which by integrality entails $\langle \mathrm{g}, \mathrm{e}\rangle \geqslant 1$. By Observation 13, (3), and (4), we have

$$\widehat{B} \geqslant \langle \mathrm{g}, \mathrm{v}\rangle = \langle \mathrm{g}, \mathrm{u}\rangle = \sum_{\mathrm{d}\in\mathcal{D}} \lambda_{\mathrm{d}} \cdot \langle \mathrm{g}, \mathrm{d}\rangle \geqslant \lambda_{\mathrm{e}} \cdot \langle \mathrm{g}, \mathrm{e}\rangle \geqslant \lambda_{\mathrm{e}} = \mu_{\mathrm{e}}.$$

Together with the straightforward inequality $\mu_{\mathrm{d}} \leqslant K \leqslant \widehat{B}$ for all $\mathrm{d} \in \mathcal{D}_{\mathcal{G}}$, we conclude that

$$\mu_{\mathrm{d}} \leqslant \widehat{B} \qquad \text{for all } \mathrm{d} \in \mathcal{D}. \tag{5}$$

Let

$$\mathrm{u}' \coloneqq \sum_{\mathrm{d}\in\mathcal{D}} \mu_{\mathrm{d}} \cdot \mathrm{d}.$$

Clearly $\mathrm{u}' \in \mathsf{intCone}(\mathcal{D})$. Our goal is to show that Lemma 16 can be applied to $\mathrm{v} - \mathrm{u}'$.

First, consider any $\mathrm{f} \in \mathcal{F} - \mathcal{G}$. Then by (3) and (5), we have

$$\begin{aligned}\langle \mathrm{f}, \mathrm{v} - \mathrm{u}'\rangle &= \langle \mathrm{f}, \mathrm{v}\rangle - \langle \mathrm{f}, \mathrm{u}'\rangle > \widehat{M}\widehat{B} - \sum_{\mathrm{d}\in\mathcal{D}} \mu_{\mathrm{d}} \cdot \langle \mathrm{f}, \mathrm{d}\rangle \\ &\geqslant \widehat{M}\widehat{B} - |\mathcal{D}| \cdot \max_{\mathrm{f}\in\mathcal{F}} \|\mathrm{f}\|_1 \cdot \max_{\mathrm{d}\in\mathcal{D}} \|\mathrm{d}\|_\infty \cdot \widehat{B} = M\widehat{B} \geqslant M. \end{aligned} \tag{6}$$

Next, consider any $\mathrm{g} \in \mathcal{G}$. Then

$$\langle \mathrm{g}, \mathrm{v} - \mathrm{u}'\rangle = \sum_{\mathrm{d}\in\mathcal{D}} (\lambda_{\mathrm{d}} - \mu_{\mathrm{d}}) \cdot \langle \mathrm{g}, \mathrm{d}\rangle = \sum_{\mathrm{d}\in\mathcal{D}_{\mathcal{G}}} (\lambda_{\mathrm{d}} - \mu_{\mathrm{d}}) \cdot \langle \mathrm{g}, \mathrm{d}\rangle + \sum_{\mathrm{d}\in\mathcal{D}-\mathcal{D}_{\mathcal{G}}} (\lambda_{\mathrm{d}} - \mu_{\mathrm{d}}) \cdot \langle \mathrm{g}, \mathrm{d}\rangle. \tag{7}$$

Note that for $\mathrm{d} \in \mathcal{D} - \mathcal{D}_{\mathcal{G}}$ we have $\lambda_{\mathrm{d}} = \mu_{\mathrm{d}}$, while for $\mathrm{d} \in \mathcal{D}_{\mathcal{G}}$ we have $\langle \mathrm{g}, \mathrm{d} \rangle = 0$. So both summands on the right hand side of (7) are equal to 0, implying that

$$\langle \mathrm{g}, \mathrm{v} - \mathrm{u}' \rangle = 0 \qquad \text{for all } \mathrm{g} \in \mathcal{G}. \tag{8}$$

Observe that since $\mathcal{F}$ is the dual representation of $\mathsf{cone}(\mathcal{D})$, (6) and (8) together imply that

$$\mathrm{v} - \mathrm{u}' \in \mathsf{cone}(\mathcal{D}). \tag{9}$$

Then by combining (8), (9), and Lemma 14 we conclude that

$$\mathrm{v} - \mathrm{u}' \in \mathsf{cone}(\mathcal{D}_{\mathcal{G}}). \tag{10}$$

Finally, observe that for every $\mathrm{d} \in \mathcal{D}$, $\lambda_{\mathrm{d}} - \mu_{\mathrm{d}}$ is an integer divisible by K. Therefore, $\mathrm{u} - \mathrm{u}' = K \cdot \mathrm{w}$ for some integer vector w, implying that $\mathrm{u} - \mathrm{u}' \in \Lambda_0^K$. As $\mathrm{u}, \mathrm{v} \in \Lambda_{\mathrm{r}}^B$ and K divides B, we also have $\mathrm{v} - \mathrm{u} \in \Lambda_0^K$. Combining these two observations yields

$$\mathrm{v} - \mathrm{u}' \in \Lambda_0^K. \tag{11}$$

Noting that $0 \in \mathsf{lattice}(\mathcal{D}_{\mathcal{G}}) \cap \Lambda_0^K$ and K is divisible by $K_{\mathcal{G}}$, by applying Lemma 18 to $\mathcal{D}_{\mathcal{G}}$ we infer that

$$\mathsf{span}(\mathcal{D}_{\mathcal{G}}) \cap \Lambda_0^K \subseteq \mathsf{lattice}(\mathcal{D}_{\mathcal{G}}).$$

Combining this with (10) and (11) yields

$$\mathrm{v} - \mathrm{u}' \in \mathsf{lattice}(\mathcal{D}_{\mathcal{G}}). \tag{12}$$

Now, assertions (6), (10), and (12) show that we can use Lemma 16 to conclude that

$$\mathrm{v} - \mathrm{u}' \in \mathsf{intCone}(\mathcal{D}_{\mathcal{G}}).$$

Since $\mathrm{u}' = \sum_{\mathrm{d} \in \mathcal{D}} \mu_{\mathrm{d}} \cdot \mathrm{d}$ and $\langle \mu_{\mathrm{d}} \colon \mathrm{d} \in \mathcal{D} \rangle$ are nonnegative integers, we have $\mathrm{u}' \in \mathsf{intCone}(\mathcal{D}_{\mathcal{G}})$ as well. So as a sum of two vectors from $\mathsf{intCone}(\mathcal{D}_{\mathcal{G}})$, v also belongs to $\mathsf{intCone}(\mathcal{D}_{\mathcal{G}})$, and we are done. □

For every $\mathcal{G} \subseteq \mathcal{F}$, let

$$\mathcal{R}_{\mathcal{G}} \coloneqq \{\mathrm{v} \in \mathcal{R} \mid \mathsf{Tight}(\mathrm{v}) = \mathcal{G}\}.$$

In other words, $\mathcal{R}_{\mathcal{G}}$ comprises all vectors in $\mathcal{R}$ for which $\mathcal{G}$ is exactly the set of tight vectors in $\mathcal{F}$. Thus, $\{\mathcal{R}_{\mathcal{G}} \colon \mathcal{G} \subseteq \mathcal{F}\}$ is a partition of $\mathcal{R}$.

Let $\mathscr{L}$ be the family of subsets of $\mathcal{F}$ consisting of all $\mathcal{G} \subseteq \mathcal{F}$ such that $\mathcal{R}_{\mathcal{G}} \cap \mathcal{Z} \neq \emptyset$. Further, let $\mathscr{L}^{\downarrow}$ be the downward closure of $\mathscr{L}$: a set $\mathcal{G} \subseteq \mathcal{F}$ belongs to $\mathscr{L}^{\downarrow}$ if and only if there exists $\mathcal{G}' \supseteq \mathcal{G}$ such that $\mathcal{G}' \in \mathscr{L}$. The following statement is an immediate corollary of Claim 21.

Claim 22. Suppose $\mathcal{G} \subseteq \mathcal{F}$. If $\mathcal{G} \in \mathscr{L}^{\downarrow}$ then $\mathcal{R}_{\mathcal{G}} \subseteq \mathcal{Z}$, and if $\mathcal{G} \notin \mathscr{L}^{\downarrow}$ then $\mathcal{R}_{\mathcal{G}} \cap \mathcal{Z} = \emptyset$. Consequently,

$$\mathcal{Z} = \bigcup_{\mathcal{G} \in \mathscr{L}^{\downarrow}} \mathcal{R}_{\mathcal{G}}.$$

Next, we show that the characterization of Claim 22 can be expressed through linear inequalities.

Claim 23. For every $\mathrm{v} \in \Lambda_{\mathrm{r}}^{B}$, the following conditions are equivalent:
(1) $\mathrm{v} \in \mathsf{intCone}(\mathcal{D})$; and
(2) $\mathrm{v} \in \mathsf{cone}(\mathcal{D})$ and

$$\sum_{\mathrm{g} \in \mathcal{G}} \langle \mathrm{g}, \mathrm{v} \rangle \geqslant 1 + \sum_{\mathrm{g} \in \mathcal{G}} p_{\mathrm{g}} \qquad \text{for all } \mathcal{G} \subseteq \mathcal{F} \text{ such that } \mathcal{G} \notin \mathscr{L}^{\downarrow}.$$

Proof. We first prove implication (1)⇒(2). Since $\mathrm{v} \in \Lambda_{\mathrm{r}}^{B} \cap \mathsf{intCone}(\mathcal{D}) = \mathcal{Z}$, by Claim 22 we have that $\mathrm{v} \in \mathcal{R}_{\mathcal{G}}$ for some $\mathcal{G} \in \mathscr{L}^{\downarrow}$. In particular, for every $\mathcal{G}' \subseteq \mathcal{F}$ with $\mathcal{G}' \notin \mathscr{L}^{\downarrow}$, $\mathcal{G}'$ is not a subset of $\mathcal{G}$, implying that there exists some $\mathrm{h} \in \mathcal{G}' - \mathcal{G}$. As $\mathcal{G} = \mathsf{Tight}(\mathrm{v})$, h is not tight for v, hence $\langle \mathrm{h}, \mathrm{v} \rangle \geqslant p_{\mathrm{h}} + B$ by Claim 20. Hence, by Claim 20 again,

$$\sum_{\mathrm{g} \in \mathcal{G}'} \langle \mathrm{g}, \mathrm{v} \rangle \geqslant B + \sum_{\mathrm{g} \in \mathcal{G}'} p_{\mathrm{g}} \geqslant 1 + \sum_{\mathrm{g} \in \mathcal{G}'} p_{\mathrm{g}};$$

and this holds for each $\mathcal{G}' \subseteq \mathcal{F}$ with $\mathcal{G}' \notin \mathscr{L}^{\downarrow}$. As $\mathrm{v} \in \mathsf{intCone}(\mathcal{D})$ entails $\mathrm{v} \in \mathsf{cone}(\mathcal{D})$, this proves (2).

We now move to the implication (2)⇒(1). Since $\mathrm{v} \in \Lambda_{\mathrm{r}}^{B} \cap \mathsf{cone}(\mathcal{D}) = \mathcal{R}$, by Claim 22 it suffices to show that the unique $\mathcal{G} \subseteq \mathcal{F}$ for which $\mathrm{v} \in \mathcal{R}_{\mathcal{G}}$ satisfies $\mathcal{G} \in \mathscr{L}^{\downarrow}$. Note $\mathrm{v} \in \mathcal{R}_{\mathcal{G}}$ is equivalent to $\mathcal{G} = \mathsf{Tight}(\mathrm{v})$. Therefore $\langle \mathrm{g}, \mathrm{v} \rangle = p_{\mathrm{g}}$ for all $\mathrm{g} \in \mathcal{G}$, implying that

$$\sum_{\mathrm{g} \in \mathcal{G}} \langle \mathrm{g}, \mathrm{v} \rangle = \sum_{\mathrm{g} \in \mathcal{G}} p_{\mathrm{g}}.$$

Hence we necessarily must have $\mathcal{G} \in \mathscr{L}^{\downarrow}$, for otherwise (2) would not be satisfied. □

We may now define $\mathcal{Q}$ to be the set of the following pairs:
- $(\mathrm{f}, 0)$ for all $\mathrm{f} \in \mathcal{F}$; and
- $\left(\sum_{\mathrm{g} \in \mathcal{G}} \mathrm{g}, 1 + \sum_{\mathrm{g} \in \mathcal{G}} p_{\mathrm{g}}\right)$ for all $\mathcal{G} \subseteq \mathcal{F}$ with $\mathcal{G} \notin \mathscr{L}^{\downarrow}$.

As $\mathcal{F}$ is a dual representation of $\mathsf{cone}(\mathcal{D})$, we have $\mathsf{cone}(\mathcal{D}) = \{\mathrm{v} \in \mathbb{R}^{\mathrm{t}} \mid \langle \mathrm{f}, \mathrm{v} \rangle \geqslant 0 \text{ for all } \mathrm{f} \in \mathcal{F}\}$. Hence, Claim 23 directly implies that

$$\Lambda_{\mathrm{r}}^{B} \cap \mathsf{intCone}(\mathcal{D}) = \Lambda_{\mathrm{r}}^{B} \cap \left\{\mathrm{v} \in \mathbb{R}^{\mathrm{t}} \mid \langle \mathrm{q}, \mathrm{v} \rangle \geqslant a \text{ for all } (\mathrm{q}, a) \in \mathcal{Q}\right\},$$

as required.

It remains to argue that the set $\mathcal{Q}$ can be computed given $\mathcal{D}$ and r. For this, it suffices to distinguish which sets $\mathcal{G} \subseteq \mathcal{F}$ belong to $\mathscr{L}$ and which not, because based on this knowledge we may compute $\mathscr{L}^{\downarrow}$ and then construct $\mathcal{Q}$ right from the definition. (Recall here that by Theorem 12, $\mathcal{F}$ can be computed from $\mathcal{D}$.) Testing whether given $\mathcal{G} \subseteq \mathcal{F}$ belongs to $\mathscr{L}$ boils down to verifying whether there exists $\mathrm{v} \in \mathbb{Z}^{\mathrm{t}}$ such that
- $\mathrm{v} \in \Lambda_{\mathrm{r}}^{B}$, or equivalently, $\mathrm{v} = B \cdot \mathrm{w} + \mathrm{r}$ for some $\mathrm{w} \in \mathbb{Z}^{\mathrm{t}}$;

- $\langle \mathrm{g}, \mathrm{v} \rangle = p_{\mathrm{g}}$ for all $\mathrm{g} \in \mathcal{G}$; and
- $\langle \mathrm{g}, \mathrm{v} \rangle \geqslant p_{\mathrm{g}} + 1$ for all $\mathrm{g} \in \mathcal{F} - \mathcal{G}$; and
- $\mathrm{v} \in \mathsf{lattice}(\mathcal{D})$, or equivalently, there exist nonnegative integer coefficients $\langle \lambda_{\mathrm{d}} \colon \mathrm{d} \in \mathcal{D} \rangle$ such that $\mathrm{v} = \sum_{\mathrm{d} \in \mathcal{D}} \lambda_{\mathrm{d}} \cdot \mathrm{d}$.

These conditions form an integer program with $2|\mathrm{t}| + |\mathcal{D}|$ variables: $|\mathrm{t}|$ variables for v, $|\mathrm{t}|$ variables for w, and $|\mathcal{D}|$ variables for the coefficients $\langle \lambda_{\mathrm{d}} \colon \mathrm{d} \in \mathcal{D} \rangle$. Hence we may just check the feasibility of this program using any of the standard algorithms for integer programming, e.g., the algorithm of Kannan [28]. □

We remark that while the statement of Theorem 19 does not specify any concrete upper bound on the value of B, it is not hard to trace all the estimates used throughout the reasoning to see that B is bounded by an elementary function (that is, a constant-height tower of exponents) of the relevant parameters $|\mathrm{t}|$, $|\mathcal{D}|$, and $\max_{\mathrm{d} \in \mathcal{D}} \|\mathcal{D}\|_\infty$. We did not attempt to optimize the value of B in our proof, hence finding tighter estimates is left to future work.

4.4 Algorithm for two-stage stochastic integer programming

With all the tools prepared, we are ready to give a proof of Theorem 11.

Proof of Theorem 11. For each matrix D_i, let $\mathcal{D}_i$ be the set of columns of D_i. Since every member of $\mathcal{D}_i$ is a vector in $\mathbb{Z}^{\mathrm{t}}$ with all entries of absolute value at most Δ, and there are $(2\Delta + 1)^{|\mathrm{t}|}$ different such vectors, we have $|\mathcal{D}_i| \leqslant (2\Delta + 1)^{|\mathrm{t}|}$. In particular, there are at most $2^{(2\Delta+1)^{|\mathrm{t}|}}$ distinct sets $\mathcal{D}_i$.

Now, the feasibility of the program P is equivalent to the following assertion: there exist $\mathrm{u} \in \mathbb{Z}^{\mathrm{x}}_{\geqslant 0}$ such that

$$\mathrm{b}_i - A_i \mathrm{u} \in \mathsf{intCone}(\mathcal{D}_i) \qquad \text{for all } i \in I. \tag{13}$$

For each $i \in I$ apply Theorem 19 to $\mathcal{D}_i$, yielding a positive integer B_i, computable from $\mathcal{D}_i$. Let B be the least common multiple of all integers B_i for $i \in I$. Since the number of distinct sets $\mathcal{D}_i$ is bounded by $2^{(2\Delta+1)^{|\mathrm{t}|}}$, it follows that B is bounded by a computable function of Δ and $|\mathrm{t}|$.

Towards verification of assertion (13), the algorithm guesses, by iterating through all possibilities, the vector $\mathrm{r} \in \{0, 1, \ldots, B-1\}^{\mathrm{x}}$ such that $\mathrm{u}[x] \equiv \mathrm{r}[x] \bmod B$ for each $x \in \mathrm{x}$; equivalently $\mathrm{u} \in \Lambda^B_{\mathrm{r}}$. This uniquely defines, for each $i \in I$, a vector $\mathrm{r}_i \in \{0, 1, \ldots, B_i - 1\}^{\mathrm{t}}$ such that

$$(\mathrm{b}_i - A_i \mathrm{u})[t] \equiv \mathrm{r}_i[t] \mod B_i \qquad \text{for all } t \in \mathrm{t}.$$

Or equivalently, $\mathrm{b}_i - A_i \mathrm{u} \in \Lambda^{B_i}_{\mathrm{r}_i}$.

Apply the algorithm of Theorem 19 to $\mathcal{D}_i$ and r_i, yielding a set of pairs $\mathcal{Q}_i \subseteq \mathbb{Z}^{\mathrm{t}} \times \mathbb{Z}$ such that

$$\Lambda^{B_i}_{\mathrm{r}_i} \cap \mathsf{intCone}(\mathcal{D}_i) = \Lambda^{B_i}_{\mathrm{r}_i} \cap \{\mathrm{v} \in \mathbb{R}^{\mathrm{t}} \mid \langle \mathrm{q}, \mathrm{v} \rangle \geqslant a \text{ for all } (\mathrm{q}, a) \in \mathcal{Q}_i\}.$$

Hence, under the supposition that indeed $\mathrm{u} \in \Lambda^B_{\mathrm{r}}$, assertion (13) boils down to verifying the existence of $\mathrm{u} \in \mathbb{Z}^{\mathrm{x}}$ satisfying the following:

- $\mathrm{u}[x] \geqslant 0$ for all $x \in \mathrm{x}$;
- $\mathrm{u} \in \Lambda^B_{\mathrm{r}}$, or equivalently, $\mathrm{u} = B \cdot \mathrm{w} + \mathrm{r}$ for some $\mathrm{w} \in \mathbb{Z}^{\mathrm{x}}$;
- for each $i \in I$ and each $(\mathrm{q}, a) \in \mathcal{Q}_i$, we have

$$\langle \mathrm{q}, \mathrm{b}_i - A_i \mathrm{u} \rangle \geqslant a.$$

The conditions above form an integer program with $2|\mathrm{x}|$ variables ($|\mathrm{x}|$ variables for u and $|\mathrm{x}|$ variables for w) whose total bitsize is bounded by $g(\Delta, |\mathrm{t}|) \cdot \|P\|$ for some computable function g. Hence, we can solve this program (and thus verify the existence of a suitable u) in time $h(\Delta, |\mathrm{x}|, |\mathrm{t}|) \cdot \|P\|$ for a computable h using, for instance, the algorithm of Kannan [28]. Iterating through all $\mathrm{r} \in \{0, 1, \ldots, B-1\}^{\mathrm{x}}$ adds only a multiplicative factor that depends in a computable manner on $\Delta, |\mathrm{x}|, |\mathrm{t}|$, yielding in total a time complexity bound of $f(\Delta, |\mathrm{x}|, |\mathrm{t}|) \cdot \|P\|$ for a computable function f, as claimed. □

5 n-fold integer programming

Our main result for n-fold integer programming is presented in the following statement, which is just Theorem 2 with adjusted notation.

Theorem 24. An instance $P = (C, \mathrm{a}, \langle D_i, \mathrm{b}_i, \mathrm{c}_i : i \in I \rangle)$ of Uniform n-fold ILP Optimization can be solved in time $f(\Delta, |\mathrm{y}|, |\mathrm{t}|) \cdot \|P\|^{\mathcal{O}(1)}$ for some computable function f, where $\Delta = \max_{i \in I} \|D_i\|_\infty$.

Again, importantly, we do not impose any upper bound on the absolute values of the entries of the matrix C. Note also that as mentioned in Section 1, the considered parameters do not include $|\mathrm{s}|$, the number of linking constraints, but do include $|\mathrm{y}|$, the number of local variables in each block.

The remainder of this section is devoted to the proof of Theorem 24.

5.1 Graver bases

One ingredient that we will repeatedly use in our proofs is the notion of the Graver basis of a matrix, which consists of $\sqsubseteq$-minimal integral elements of the kernel. Formally, the integer kernel of a matrix $D \in \mathbb{Z}^{\mathrm{y} \times \mathrm{t}}$, denoted $\ker^{\mathbb{Z}}(D)$, is the set of all integer vectors $\mathrm{u} \in \mathbb{Z}^{\mathrm{t}}$ such that $D\mathrm{u} = 0$. The Graver basis of D is the set $\mathsf{Graver}(D)$ of all $\sqsubseteq$-minimal non-zero elements of $\ker^{\mathbb{Z}}(D)$. In other words, a vector $\mathrm{u} \in \ker^{\mathbb{Z}}(D) - \{0\}$ belongs to $\mathsf{Graver}(D)$ if there is no $\mathrm{u}' \sqsubseteq \mathrm{u}$, $\mathrm{u}' \notin \{0, \mathrm{u}\}$, such that $\mathrm{u}' \in \ker^{\mathbb{Z}}(D)$ as well.

Note that since $\mathsf{Graver}(D)$ is an antichain in the $\sqsubseteq$ order, which is a well quasi-order on $\mathbb{Z}^{\mathrm{t}}$ by Dickson's Lemma, $\mathsf{Graver}(D)$ is always finite. There are actually many known bounds on the norms of the elements of the Graver basis under different assumptions about the matrix. We will use the following one tailored to matrices with few rows.

Lemma 25 (Lemma 2 of [19]). Let $D \in \mathbb{Z}^{\mathrm{y} \times \mathrm{t}}$ be an integer matrix, and let $\Delta = \|D\|_\infty$. Then for every $\mathrm{g} \in \mathsf{Graver}(D)$, it holds that

$$\|\mathrm{g}\|_1 \leqslant (2|\mathrm{t}|\Delta + 1)^{|\mathrm{t}|}$$

We will also use the fact that every solution to an integer program can be decomposed into some solution of bounded norm and a multiset consisting of elements of the Graver basis. This fact was observed by Pottier in [44, Corollary 1], we recall his proof for completeness.

Lemma 26. Let $D \in \mathbb{Z}^{y \times t}$ be an integer matrix and $b \in \mathbb{Z}^t_{\geqslant 0}$ be an integer vector. Then for every $w \in \mathbb{Z}^y_{\geqslant 0}$ such that $Dw = b$, there exists a vector $\widehat{w} \in \mathbb{Z}^y_{\geqslant 0}$ and a multiset $\mathcal{G}$ consisting of nonnegative elements of $\mathsf{Graver}(D)$ such that

$$D\widehat{w} = b, \qquad \|\widehat{w}\| \leqslant (2|t|(\|D\|_\infty + \|b\|_\infty) + 1)^{|t|}, \qquad \text{and} \qquad w = \widehat{w} + \sum_{g \in \mathcal{G}} g.$$

Proof. Let z be a fresh variable that does not belong to y and let $z := y \cup \{z\}$. Further, let $D' \in \mathbb{Z}^{z \times t}$ be the matrix obtained from D by adding column $D'[z] = -b$, and $w' \in \mathbb{Z}^z_{\geqslant 0}$ be the vector obtained from w by adding the entry $w'[z] = 1$. Observe that $Dw = b$ entails $D'w' = 0$, which means that $w' \in \ker^{\mathbb{Z}}(D')$. Every element of the integer kernel can be decomposed into a sign-compatible sum of elements of the Graver basis; see e.g. [16, Lemma 3.2.3]. Therefore, there is a multiset $\mathcal{G}'$ consisting of nonnegative elements of $\mathsf{Graver}(D')$ such that $\sum_{g' \in \mathcal{G}'} g' = w'$. Due to the sign-compatibility, exactly one element $\widehat{w}' \in \mathcal{G}'$ has $\widehat{w}[z] = 1$, and all the other elements $g' \in \mathcal{G}' - \{\widehat{w}'\}$ satisfy $g'[z] = 0$. We can now obtain $\widehat{w}$ and $\mathcal{G}$ from $\widehat{w}'$ and $\mathcal{G}' - \{\widehat{w}'\}$, respectively, by stripping every vector from the coordinate corresponding to variable z (formally, restricting the domain to y). As $g'[z] = 0$ for all $g' \in \mathcal{G}' - \{\widehat{w}'\}$, it follows that all elements of $\mathcal{G}$ belong to $\mathsf{Graver}(D)$. Finally, from Lemma 25 applied to D' we conclude that $\|\widehat{w}\|_\infty \leqslant \|\widehat{w}'\|_\infty \leqslant (2|t|(\|D\|_\infty + \|b\|_\infty) + 1)^{|t|}$. □

5.2 Decomposing bricks

As explained in Section 2, the key idea in the proof of Theorem 24 is to decompose right-hand sides of the program (that is, vectors b_i) into smaller and smaller vectors while preserving the optimum value of the program. The next lemma is the key observation that provides a single step of the decomposition.

Lemma 27 (Brick Decomposition Lemma). There exists a function $g(k, \Delta) \in 2^{(k\Delta)^{\mathcal{O}(k)}}$ such that the following holds. Let $D \in \mathbb{Z}^{y \times t}$ be a matrix and $b \in \mathbb{Z}^t$ be a vector such that $\|b\|_\infty > g(k, \Delta)$, where $k = |t|$ and $\Delta = \|D\|_\infty$. Then there are non-zero vectors $b', b'' \in \mathbb{Z}^t$ such that the following conditions hold:

- $b', b'' \sqsubseteq b$ and $b = b' + b''$; and
- for every $v \in \mathbb{Z}^y_{\geqslant 0}$ satisfying $Dv = b$, there exist $v', v'' \in \mathbb{Z}^y_{\geqslant 0}$ such that

$$v = v' + v'', \qquad Dv' = b', \qquad \text{and} \qquad Dv'' = b''.$$

Throughout this section, for a multiset V of vectors or numbers, we denote by $\sum V$ the sum of the elements in V.

The crucial ingredient to prove Lemma 27 is the following result of Klein [29]. We use the variant from Cslovjecsek et al. [13], which compared to the original

version of Klein [29] has better bounds and applies to vectors with possibly negative coefficients.

Lemma 28 (Klein Lemma, Theorem 4.1 of [13] for $\epsilon = 1$ and with adjusted notation). Let t be a tuple of variables with $|\mathrm{t}| = k$ and let $T_1, \ldots, T_n$ be non-empty multisets of vectors in $\mathbb{Z}^{\mathrm{t}}$ such that for all $i \in \{1, \ldots, n\}$ and $\mathrm{u} \in T_i$, we have $\|\mathrm{u}\|_\infty \leqslant \Delta$, and the sum of all elements in each multiset is the same:

$$\sum T_1 = \sum T_2 = \ldots = \sum T_n.$$

Then there exist non-empty submultisets $S_1 \subseteq T_1, S_2 \subseteq T_2, \ldots, S_n \subseteq T_n$, each of size bounded by $2^{\mathcal{O}(k\Delta)^k}$, such that

$$\sum S_1 = \sum S_2 = \ldots = \sum S_n.$$

We also need the following lemma about partitioning a multiset of vectors into small submultisets with sign-compatible sums.

Lemma 29. Let U be a multiset of vectors in $\mathbb{Z}^{\mathrm{t}}$ with $\|\mathrm{u}\|_\infty \leqslant \Delta$ for all $\mathrm{u} \in U$, where t is a tuple of variables with $|\mathrm{t}| = k$. Further, let $\mathrm{b} = \sum U$. Then one can partition U into a collection of non-empty submultisets $\langle U_i \colon i \in I \rangle$ so that for every $i \in I$, we have $|U_i| \leqslant (2k\Delta + 1)^k$ and $\sum U_i \sqsubseteq \mathrm{b}$.

Proof. Let W be a multiset of vectors in $\mathbb{Z}^{\mathrm{t}}$ such that

- $\mathrm{w} \sqsubseteq -\mathrm{b}$ and $\|\mathrm{w}\|_\infty \leqslant 1$ for each $\mathrm{w} \in W$, and
- $\sum \mathrm{w} = -\mathrm{b}$.

Let x and y be tuples of variables enumerating the elements of U and W, respectively, so that we can set up a matrix $A \in \mathbb{Z}^{(\mathrm{x} \cup \mathrm{y}) \times \mathrm{t}}$ with the elements of U as the columns corresponding to x and the elements of W as the columns corresponding to y. Thus, we have

$$A1 = 0.$$

By Lemma 25, there exists $\mathrm{g} \in \mathsf{Graver}(A)$ such that

$$\|\mathrm{g}\|_1 \leqslant (2k\Delta + 1)^k \qquad \text{and} \qquad \mathrm{g} \sqsubseteq 1.$$

Let U_1 be the multisets of those elements of U that correspond to the variables $x \in \mathrm{x}$ with $\mathrm{g}(x) = 1$. Similarly, let W_1 be the multiset of those elements of W that correspond to the variables $y \in \mathrm{y}$ with $\mathrm{g}(y) = 1$. Thus, we have $|U_1| + |W_1| = \|\mathrm{g}\|_1 \leqslant (2k\Delta + 1)^k$. Moreover, as $A\mathrm{g} = 0$, $\sum W = -\mathrm{b}$, and $\mathrm{w} \sqsubseteq -\mathrm{b}$ for each $\mathrm{w} \in W_1$, we have

$$\sum U_1 = -\sum W_1 \sqsubseteq \mathrm{b}.$$

Therefore, we can set U_1 to be the first multiset in the sought collection. It now remains to apply the same reasoning inductively to the multiset $U - U_1$ in order to extract the next elements of the collection, until the multiset under consideration becomes empty. □

We are now ready to prove Lemma 27.

Proof of Lemma 27. Let $\Xi \in 2^{\mathcal{O}(k\Delta)^k}$ be the bound provided by Lemma 28 for parameters k and Δ.

Call a vector $\mathrm{v} \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ a solution if $D\mathrm{v} = \mathrm{b}$. Further, a solution v is minimal if it is conformally minimal among the solutions: there is no solution v' such that $\mathrm{v}' \neq \mathrm{v}$ and $\mathrm{v}' \sqsubseteq \mathrm{v}$. Let V be the set of all minimal solutions. Note that V is an antichain in $\sqsubseteq$, so as $\sqsubseteq$ is a well quasi-order on $\mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ by Dickson's Lemma, it follows that V is finite. If V is empty, then there are no solutions and there is noting to prove, so assume otherwise.

For each $\mathrm{v} \in V$, construct a multiset of vectors T_{v} as follows: for each $y \in \mathrm{y}$, include $\mathrm{v}[y]$ copies of the column $D[y]$ (i.e., the column of D corresponding to variable y) in T_{v}. Thus, we obtain a finite collection of multisets $\langle T_{\mathrm{v}} \colon \mathrm{v} \in V \rangle$. By construction, we have

$$\sum T_{\mathrm{v}} = D\mathrm{v} = \mathrm{b} \qquad \text{for all } \mathrm{v} \in V.$$

Note that all vectors in all multisets T_{v} have all entries bounded in absolute value by Δ. In the sequel, we will use the following claim a few times.

Claim 30. Suppose $\mathrm{v} \in V$ and $A \subseteq T_{\mathrm{v}}$ is a submultiset such that $\sum A = 0$. Then A is empty.

Proof. Let $\mathrm{w} \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ be the vector of multiplicities in which the columns of D appear in A: for $y \in \mathrm{y}$, $\mathrm{w}[y]$ is the number of times $D[y]$ appears in A. Clearly, $\mathrm{w} \sqsubseteq \mathrm{v}$ and $\sum A = 0$ implies that $D\mathrm{w} = 0$. Then $D(\mathrm{v} - \mathrm{w}) = \mathrm{b}$ and $\mathrm{v} - \mathrm{w} \sqsubseteq \mathrm{v}$ as well. This is a contradiction with the minimality of v unless $\mathrm{w} = 0$, or equivalently, A is empty. □

As V is finite, we may apply Lemma 28 to multisets $\langle T_{\mathrm{v}} \colon \mathrm{v} \in V \rangle$. In this way, we obtain submultisets $\langle S_{\mathrm{v}} \subseteq T_{\mathrm{v}} \colon \mathrm{v} \in V \rangle$, each of size at most Ξ, such that

$$\sum S_{\mathrm{v}} = \sum S_{\mathrm{v}'} \qquad \text{for all } \mathrm{v}, \mathrm{v}' \in V.$$

Consider multisets $T'_{\mathrm{v}} \coloneqq T_{\mathrm{v}} - S_{\mathrm{v}}$ for $\mathrm{v} \in V$. Observe that the multisets T'_{v} have again the same sums. Moreover, if some T'_{v} is empty, then $\sum T'_{\mathrm{v}'} = 0$ for every $\mathrm{v}' \in V$, which by Claim 30 implies that $T'_{\mathrm{v}'}$ is empty as well. It follows that either all multisets $\langle T'_{\mathrm{v}} \colon \mathrm{v} \in V \rangle$ are empty, or all of them are non-empty.

If all multisets $\langle T'_{\mathrm{v}} \colon \mathrm{v} \in V \rangle$ are non-empty, then we may apply the same argument to them again, and thus extract suitable non-empty submultisets $\langle S'_{\mathrm{v}} \subseteq T'_{\mathrm{v}} \colon \mathrm{v} \in V \rangle$ with the same sum. By performing this reasoning iteratively until all multisets become empty (which occurs simultaneously due to Claim 30), we find a collection of partitions

$$\left\langle \left\langle S^i_{\mathrm{v}} \colon i \in I \right\rangle : \mathrm{v} \in V \right\rangle$$

for some index set I such that:

- $\left\langle S^i_{\mathrm{v}} \colon i \in I \right\rangle$ is a partition of T_{v} for each $\mathrm{v} \in V$;

- S^i_{v} is non-empty and $|S^i_{\mathrm{v}}| \leqslant \Xi$ for all $i \in I$ and $\mathrm{v} \in V$; and
- $\sum S^i_{\mathrm{v}} = \sum S^i_{\mathrm{v}'}$ for all $i \in I$ and $\mathrm{v}, \mathrm{v}' \in V$.

For $i \in I$, let p_i be the common sum of multisets S^i_{v} for $\mathrm{v} \in V$, that is,

$$\mathrm{p}_i = \sum S^i_{\mathrm{v}} \qquad \text{for all } \mathrm{v} \in V.$$

Note that $\sum_{i \in I} \mathrm{p}_i = \mathrm{b}$ and $\|\mathrm{p}_i\|_\infty \leqslant \Delta\Xi =: \Delta'$ for all $i \in I$.

We now apply Lemma 29 to the collection of vectors $\langle \mathrm{p}_i \colon i \in I \rangle$. This yields a partition $\langle I_j \colon j \in J \rangle$ of I with some index set J such that for each $j \in J$, we have

$$|I_j| \leqslant (2k\Delta' + 1)^k \leqslant 2^{(k\Delta)^{\mathcal{O}(k)}} \qquad \text{and} \qquad \sum_{i \in I_j} \mathrm{p}_i \sqsubseteq \mathrm{b}.$$

By setting $g(k, \Delta) \in 2^{(k\Delta)^{\mathcal{O}(k)}}$ large enough, we may guarantee that $|I_j| \cdot \Delta \leqslant g(k, \Delta)$ for all $j \in J$. Since we assumed that $\|b\|_\infty > g(k, d)$, we conclude that $\|b\|_\infty > \sum_{i \in I_j} \mathrm{p}_i$ for all $j \in J$, and consequently it must be the case that $|J| \geqslant 2$.

Let $\{J', J''\}$ be any partition of J with both J' and J'' being non-empty. Define

$$I' \coloneqq \bigcup_{j \in J'} I_j \qquad \text{and} \qquad I'' \coloneqq \bigcup_{j \in J''} I_j.$$

Further, we set

$$\mathrm{b}' \coloneqq \sum_{i \in I'} \mathrm{p}_i \qquad \text{and} \qquad \mathrm{b}'' \coloneqq \sum_{i \in I''} \mathrm{p}_i.$$

As $\sum_{i \in I} \mathrm{p}_i = \mathrm{b}$ and $\mathrm{p}_i \sqsubseteq \mathrm{b}$ for all $i \in I$, it follows that $\mathrm{b}' + \mathrm{b}'' = \mathrm{b}$ and $\mathrm{b}', \mathrm{b}'' \sqsubseteq \mathrm{b}$. Further, from Claim 30 we have that both b' and b'' are non-zero.

Consider any $\mathrm{v} \in V$. Let

$$S'_{\mathrm{v}} \coloneqq \bigcup_{i \in I'} S^i_{\mathrm{v}} \qquad \text{and} \qquad S''_{\mathrm{v}} \coloneqq \bigcup_{i \in I''} S^i_{\mathrm{v}}.$$

Note that

$$\sum S'_{\mathrm{v}} = \mathrm{b}' \qquad \text{and} \qquad \sum S''_{\mathrm{v}} = \mathrm{b}''. \tag{14}$$

Further, let $\mathrm{v}' \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ be the vector of multiplicities in which the columns of D appear in S'_{v}: for $y \in \mathrm{y}$, $\mathrm{v}'[y]$ is the number of times $D[y]$ appears in S'_{v}. Define v'' analogously for S''_{v}. Since $\{S'_{\mathrm{v}}, S''_{\mathrm{v}}\}$ is a partition of T_{v}, we have $\mathrm{v}' + \mathrm{v}'' = \mathrm{v}$ and $\mathrm{v}', \mathrm{v}'' \sqsubseteq \mathrm{v}$. Moreover, from (14) it follows that

$$D\mathrm{v}' = \mathrm{b}' \qquad \text{and} \qquad D\mathrm{v}'' = \mathrm{b}''.$$

The above reasoning already settles the second condition from the lemma statement for every minimal solution. So consider now any solution $\mathrm{v} \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$. Then there exists a minimal solution $\widehat{\mathrm{v}} \in V$ such that $\widehat{\mathrm{v}} \sqsubseteq \mathrm{v}$. As both v and $\widehat{\mathrm{v}}$ are solutions, we have $D(\mathrm{v} - \widehat{\mathrm{v}}) = 0$. It follows that we may simply take

$$\mathrm{v}' \coloneqq \widehat{\mathrm{v}}' + (\mathrm{v} - \widehat{\mathrm{v}}) \qquad \text{and} \qquad \mathrm{v}'' \coloneqq \widehat{\mathrm{v}}''.$$

This concludes the proof. □

From now on, we adopt the function $g(\Delta, k)$ provided by Lemma 27 in the notation.

Now that Lemma 27 is established, it is tempting to apply it iteratively: first break b into b' and b'', then break b' into two even smaller vectors, and so on. To facilitate the discussion about such decompositions, we introduce the following definition.

Definition 31. Let $D \in \mathbb{Z}^{\mathrm{y} \times \mathrm{t}}$ be a matrix and $\mathrm{b} \in \mathbb{Z}^{\mathrm{t}}$ be a vector. A faithful decomposition of b with respect to D is a collection $\langle \mathrm{b}_i \colon i \in I \rangle$ of non-zero vectors in $\mathbb{Z}^{\mathrm{t}}$ satisfying the following conditions:

- $\mathrm{b}_i \sqsubseteq \mathrm{b}$ for each $i \in I$ and $\mathrm{b} = \sum_{i \in I} \mathrm{b}_i$; and
- for every $\mathrm{v} \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ satisfying $D\mathrm{v} = \mathrm{b}$, there exist a collection of vectors $\langle \mathrm{v}_i \colon i \in I \rangle$ in $\mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ such that

$$\mathrm{v} = \sum_{i \in I} \mathrm{v}_i \qquad \text{and} \qquad D\mathrm{v}_i = \mathrm{b}_i \quad \text{for each } i \in I.$$

The order of a faithful decomposition $\langle \mathrm{b}_i \colon i \in I \rangle$ is $\max_{i \in I} \|\mathrm{b}_i\|_\infty$.

By applying Lemma 27 iteratively, we get the following:

Lemma 32. For every matrix $D \in \mathbb{Z}^{\mathrm{y} \times \mathrm{t}}$ and a non-zero vector $\mathrm{b} \in \mathbb{Z}^{\mathrm{t}}$, there exists a faithful decomposition of b with respect to D of order at most $g(\Delta, k)$, where $k = |\mathrm{t}|$ and $\Delta = \|D\|_\infty$.

Proof. Start with a faithful decomposition $\mathcal{B}$ consisting only of b. Then, as long as $\mathcal{B}$ contains a vector p with $\|\mathrm{p}\| > g(k, \Delta)$, apply Lemma 27 to p yielding suitable vectors p' and p'', and replace p with p' and p'' in $\mathcal{B}$. It is straightforward to verify that throughout this procedure $\mathcal{B}$ remains a faithful decomposition. Moreover, the size of $\mathcal{B}$ increases in each step of the procedure, while every faithful decomposition of b has cardinality bounded by $\|\mathrm{b}\|_1$. Therefore, the procedure must eventually end, yielding a faithful decomposition of order at most $g(k, \Delta)$. □

Observe that in the worst case, a faithful decomposition $\mathcal{B}$ provided by Lemma 32 can consist of as many as $\Omega(\|\mathrm{b}\|_1)$ vectors, which in our case is exponential in the size of the bit encoding of b. However, as all vectors in $\mathcal{B}$ have ℓ_∞-norm bounded in terms of k and Δ, the total number of distinct vectors is bounded by a function of k and Δ. Therefore, a faithful decomposition $\mathcal{B}$ provided by Lemma 32 can be encoded compactly using $g(k, \Delta) \cdot \log \|\mathrm{b}\|_\infty$ bits, by storing each vector present in $\mathcal{B}$ together with its multiplicity in $\mathcal{B}$, encoded in binary. In all further algorithmic discussions we will assume that this encoding scheme for faithful decompositions.

Our next goal is to prove that the faithful decomposition of Lemma 32 can be computed algorithmically, in fixed-parameter time when parameterized by $|\mathrm{y}|$, k, and Δ, assuming b is given on input in binary. The idea is to iteratively extract a large fraction of the elements of the decomposition, so that the whole process finishes after a number of steps that is logarithmic in $\|\mathrm{b}\|_\infty$. Each

extraction step will be executed using a fixed-parameter algorithm for deciding Presburger Arithmetic. Let us give a brief introduction.

Presburger Arithmetic is the first-order theory of the structure $\langle \mathbb{Z}_{\geqslant 0}, +, 0, 1, 2, \ldots \rangle$, that is, of nonnegative integers equipped with addition (treated as a binary function) and all elements of the universe as constants. More precisely, a term is an arithmetic expression using the binary function $+$, variables (meant to be evaluated to nonnegative integers), and constants (nonnegative integers). Formulas considered in Presburger Arithmetic are constructed from atomic formulas — equalities of terms — using the standard syntax of first-order logic, including standard boolean connectives, negation, and both existential and universal quantification (over nonnegative integers). The semantics is as expected. Note that while comparison is not directly present in the signature, it can be easily emulated by existentially quantifying a slack variable: for two variables x, y, the assertion $x \leqslant y$ is equivalent to $\exists_z\ x + z = y$. As usual, $\varphi(\mathrm{x})$ denotes a formula with a tuple of free variables x, while a sentence is a formula without free variables. The length of a formula $\varphi(\mathrm{x})$, denoted $\|\varphi\|$, is defined in a standard way by structural induction on the formula and the terms contained within. Here, all constants are deemed to have length 1.

Presburger [45] famously proved that Presburger Arithmetic is decidable: there is an algorithm that given a sentence φ of first-order logic over $\langle \mathbb{Z}_{\geqslant 0}, +, 0, 1, 2, \ldots \rangle$, decides whether φ is true. As observed by Koutecký and Talmon [39], known algorithms for deciding Presburger Arithmetic can be understood as fixed-parameter algorithms in the following sense.

Theorem 33 (follows from [39, Theorem 1]). Given a first-order sentence φ over the structure $\langle \mathbb{Z}_{\geqslant 0}, +, 0, 1, 2, \ldots \rangle$, with constants encoded in binary, one can decide whether φ is true in time $f(\|\varphi\|) \cdot (\log \Delta)^{\mathcal{O}(1)}$, where $\Delta \geqslant 3$ is an upper bound on all the constants present in φ and f is a computable function.

We remark that Koutecký and Talmon discuss a more general setting where formulas are written in a more concise form, including allowing a direct usage of modular arithmetics, and one considers minimization of convex functions over tuples of variables satisfying constraints expressible in Presburger Arithmetic. Nevertheless, Theorem 33, as stated above, follows readily from [39, Theorem 1]. For a proof of [39, Theorem 1], see [38, Theorem 2.2].

We now use Theorem 33 to argue that $\forall\exists$ integer programs can be solved efficiently.

Lemma 34. Suppose $A \in \mathbb{Z}^{\mathrm{x}\times\mathrm{y}}$ and $B \in \mathbb{Z}^{\mathrm{y}\times\mathrm{t}}$ are matrices and $\mathrm{d} \in \mathbb{Z}^{\mathrm{t}}$ is a vector. Then the satisfaction of the following sentence

$$\forall_{\mathrm{v}\in\mathbb{Z}^{\mathrm{y}}} \left[(B\mathrm{v} \leqslant \mathrm{d}) \implies \exists_{\mathrm{u}\in\mathbb{Z}^{\mathrm{x}}} (A\mathrm{u} \leqslant \mathrm{v})\right]$$

can be verified in time $f(\Delta, |\mathrm{x}|, |\mathrm{y}|, |\mathrm{t}|)\cdot(\log \|\mathrm{d}\|_\infty)^{\mathcal{O}(1)}$, where $\Delta = \max(\|A\|_\infty, \|B\|_\infty)$ and f is a computable function.

Proof. It suffices to combine Theorem 33 with the observation that the considered sentence can be easily rewritten to an equivalent first-order sentence over

$\langle \mathbb{Z}_{\geqslant 0}, +, 0, 1, 2, \ldots \rangle$, whose length depends only on $\Delta, |\mathrm{x}|, |\mathrm{y}|, |\mathrm{t}|$ and whose constants are bounded by $\|\mathrm{d}\|_\infty$. Indeed, every inequality present in the constraint $B\mathrm{v} \leqslant \mathrm{d}$ can be rewritten to an equality of two terms as follows:

- Every summand on the left hand side with a negative coefficient is moved to the right hand side. Also, if the constant on the right hand side is negative, it is moved to the left hand side. Thus, after this step, every side is a sum of variables multiplied by positive coefficients and positive constants.
- Every summand of the form αv, where α is a positive coefficient and $v = \mathrm{v}[y]$ for some $y \in \mathrm{y}$, is replaced by $\underbrace{v + v + \ldots + v}_{\alpha}$. Note here that $\alpha \leqslant \Delta$.
- Inequality is turned into an equality by existentially quantifying a non-negative slack variable.

We apply a similar rewriting to the constraints present in $A\mathrm{u} \leqslant \mathrm{v}$. After these steps, the sentence is a first-order sentence over $\langle \mathbb{Z}_{\geqslant 0}, +, 0, 1, 2, \ldots \rangle$. □

We remark that Lemma 34 follows also from the work of Eisenbrand and Shmonin on $\forall\exists$ integer programs; see [18, Theorem 4.2]. In [18] Eisenbrand and Shmonin only speak about polynomial-time solvability for fixed parameters, but a careful inspection of the proof shows that in fact, the algorithm works in fixed-parameter time with relevant parameters, as described in Lemma 34.

With Lemma 34 in place, we present an algorithm to compute faithful decompositions. We first prove the following statement, which shows that a large fraction of a decomposition can be extracted efficiently.

Lemma 35. Given a matrix $D \in \mathbb{Z}^{\mathrm{y} \times \mathrm{t}}$ and a non-zero vector $\mathrm{b} \in \mathbb{Z}^{\mathrm{t}}$, one can in time $f(\Delta, |\mathrm{y}|, k) \cdot (\log \|\mathrm{b}\|_\infty)^{\mathcal{O}(1)}$, where f is a computable function, $\Delta = \|D\|_\infty$, and $k = |\mathrm{t}|$, compute a faithful decomposition $\mathcal{B}_0$ of b with respect to D with the following property: the ℓ_∞-norms of all the elements of $\mathcal{B}_0$ are bounded by $g(\Delta, k)$ except for at most one element b', for which it holds that

$$\|\mathrm{b}'\|_1 \leqslant \left(1 - \frac{1}{2^{(k\Delta)^{\mathcal{O}(k)}}}\right) \cdot \|\mathrm{b}\|_1.$$

Proof. By Lemma 32, b admits a faithful decomposition $\mathcal{B}$ with respect to D such that the order of $\mathcal{B}$ is at most $\Xi := g(\Delta, k)$. Observe that there are $M := (2\Xi+1)^k$ different vectors in $\mathbb{Z}^{\mathrm{t}}$ of ℓ_∞-norm at most Ξ, hence also at most M different vectors present in $\mathcal{B}$. Therefore, there exists a vector b_0 appearing in $\mathcal{B}$ with multiplicity $\alpha \geqslant 1$ such that

$$\alpha \cdot \|\mathrm{b}_0\|_1 \geqslant \frac{1}{M} \cdot \|\mathrm{b}\|_1.$$

Let α' be the largest power of two such that $\alpha' \leqslant \alpha$. Then we have

$$\alpha' \cdot \|\mathrm{b}_0\|_1 \geqslant \frac{1}{2M} \cdot \|\mathrm{b}\|_1. \tag{15}$$

The algorithm guesses, by trying all possibilities, vector b_0 and power of two α'. Note that there are at most $M \leqslant 2^{(k\Delta)^{\mathcal{O}(k)}}$ choices for b_0 and at most $\log \|\mathrm{b}\|_1$ choices for α', as every faithful decomposition of b consists of at most $\|\mathrm{b}\|_1$ vectors. Having guessed, b_0 and α', we define decomposition $\mathcal{B}_0$ to consist of

- α' copies of the vector b_0, and
- vector $\mathrm{b}' \coloneqq \mathrm{b} - \alpha'\mathrm{b}_0$.

As $\mathcal{B}$ is a faithful decomposition, it follows that $\mathcal{B}_0$ defined in this way is a faithful decomposition as well, provided b_0 and α' were chosen correctly. Further, we have $\|\mathrm{b}_0\| \leqslant \Xi = g(\Delta, k)$ and, by (15), also

$$\|\mathrm{b}'\|_1 \geqslant \left(1 - \frac{1}{2M}\right) \cdot \|\mathrm{b}\|_1.$$

Note that $2M = 2 \cdot (2\Xi + 1)^k = 2 \cdot (2g(\Delta, k) + 1)^k \in 2^{(k\Delta)^{\mathcal{O}(k)}}$, as promised. So what remains to prove is that for a given choice of b_0 and α', one can efficiently verify whether $\mathcal{B}_0$ defined as above is indeed a faithful decomposition of b.

Before we proceed, observe that by Lemma 26, every solution $\mathrm{w} \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ to $D\mathrm{w} = \mathrm{b}_0$ can be decomposed as

$$\mathrm{w} = \widehat{\mathrm{w}} + \sum_{\mathrm{g} \in \mathcal{G}} \mathrm{g}, \tag{16}$$

where $\widehat{\mathrm{w}} \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ is also a solution to $D\widehat{\mathrm{w}} = \mathrm{b}_0$ that additionally satisfies $\|\widehat{\mathrm{w}}\|_\infty \leqslant (2k(\Xi + \Delta) + 1)^k$, while $\mathcal{G}$ is a multiset of nonnegative vectors belonging to the Graver basis of D. Vector $\widehat{\mathrm{w}}$ will be called the base solution for w.

With this observation, we can formulate the task of verifying faithfulness of $\mathcal{B}_0$ as a $\forall\exists$ integer program as follows. We need to verify whether for every $\mathrm{v} \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ satisfying $D\mathrm{v} = \mathrm{b}$, there is a vector $\mathrm{v}' \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ satisfying $D\mathrm{v}' = \mathrm{b}'$ and vectors $\mathrm{w}_1, \ldots, \mathrm{w}_{\alpha'} \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ satisfying $D\mathrm{w}_i = \mathrm{b}_0$ for all $i \in \{1, \ldots, \alpha'\}$ such that $\mathrm{v}' + \sum_{i=1}^{\alpha'} \mathrm{w}_i = \mathrm{v}$. Note that every vector w_i will have a decomposition (16), with some base vector $\widehat{\mathrm{w}}_i$. Hence, to verify the existence of suitable $\mathrm{v}', \mathrm{w}_1, \ldots, \mathrm{w}_{\alpha'}$, it suffices to find the following objects:

- a vector $\mathrm{v}' \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ satisfying $D\mathrm{v}' = \mathrm{b}'$;
- for every $\widehat{\mathrm{w}} \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ such that $\|\widehat{\mathrm{w}}\|_\infty \leqslant (2k(\Xi + \Delta) + 1)^k$ and $D\widehat{\mathrm{w}} = \mathrm{b}_0$, a multiplicity $\gamma_{\widehat{\mathrm{w}}} \in \mathbb{Z}_{\geqslant 0}$ signifying how many times $\widehat{\mathrm{w}}$ will appear as the base solution $\widehat{\mathrm{w}}_i$; and
- for every Graver element $\mathrm{g} \in \mathsf{Graver}(D) \cap \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$, a multiplicity $\delta_{\mathrm{g}} \in \mathbb{Z}_{\geqslant 0}$ with which g will appear in the decompositions (16) of the solutions $\langle \mathrm{w}_i \colon i \in \{1, \ldots, \alpha'\} \rangle$ in total.

Note that by Lemma 25, $\mathsf{Graver}(D)$ can be computed in time depending only on Δ, $|\mathrm{y}|$, and k. The constraints that the above variables should obey, except

for integrality and nonnegativity, are the following:

$$\begin{aligned} D\mathrm{v}' &= \mathrm{b}', \\ \mathrm{v}' + \sum_{\widehat{\mathrm{w}}} \gamma_{\widehat{\mathrm{w}}} \cdot \widehat{\mathrm{w}} + \sum_{\mathrm{g}} \delta_{\mathrm{g}} \cdot \mathrm{g} &= \mathrm{v}, \\ \sum_{\widehat{\mathrm{w}}} \gamma_{\widehat{\mathrm{w}}} &= \alpha', \end{aligned}$$

where summations over $\widehat{\mathrm{w}}$ and g go over all relevant $\widehat{\mathrm{w}}$ and g for which the corresponding variables are defined. It is easy to see that the feasibility of the integer program presented above is equivalent to the existence of suitable $\mathrm{v}', \mathrm{w}_1, \ldots, \mathrm{w}_{\alpha'}$; note here that the nonnegative elements of the Graver basis can be added to any solution of $D\mathrm{w} = \mathrm{b}_0$ without changing feasibility. As we need to verify the feasibility for every $\mathrm{v} \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ satisfying $D\mathrm{v} = \mathrm{b}$, we have effectively constructed a $\forall\exists$ integer program of the form considered in Lemma 34. It now remains to observe that the number of variables of this program as well as the absolute values of all the entries in its matrices are bounded by computable functions of Δ, $|\mathrm{y}|$, and k. So we may use the algorithm of Lemma 34 to solve this $\forall\exists$ program within the promised running time bounds. □

We now give the full algorithm, which boils down to applying Lemma 35 exhaustively.

Lemma 36. Given a matrix $D \in \mathbb{Z}^{\mathrm{y}\times\mathrm{t}}$ and a non-zero vector $\mathrm{b} \in \mathbb{Z}^{\mathrm{t}}$, one can in time $f(\Delta, |\mathrm{y}|, k) \cdot (\log \|\mathrm{b}\|_\infty)^{\mathcal{O}(1)}$, where f is a computable function, $\Delta = \|D\|_\infty$, and $k = |\mathrm{t}|$, compute a faithful decomposition $\mathcal{B}$ of b with respect to D of order at most $g(\Delta, k)$.

Proof. Again, start with a faithful decomposition $\mathcal{B}$ consisting only of b. Next, as long as $\mathcal{B}$ contains a vector b' with $\|\mathrm{b}'\| > g(\Delta, k)$, apply Lemma 35 to b', which yields a faithful decomposition $\mathcal{B}'$ of b'. Then replace b with $\mathcal{B}'$ in $\mathcal{B}$ and continue. It is easy to see that $\mathcal{B}$ remains a faithful decomposition throughout this procedure, hence once all vectors in $\mathcal{B}$ have ℓ_∞-norm bounded by $g(\Delta, k)$, one can simply output $\mathcal{B}$.

It is easy to see that at each point of the procedure, $\mathcal{B}$ contains at most one vector with ℓ_∞-norm larger than $g(\Delta, k)$. Moreover, by Lemma 35, at every iteration the ℓ_1-norm of this vector decreases by being multiplied by at most $1 - \frac{1}{2^{(k\Delta)^{\mathcal{O}(k)}}}$. It follows that the total number of iterations performed by the procedure is at most $2^{(k\Delta)^{\mathcal{O}(k)}} \cdot \log \|\mathrm{b}\|_1 = 2^{(k\Delta)^{\mathcal{O}(k)}} \cdot \log \|\mathrm{b}\|_\infty$. Since by Lemma 35 every iteration takes time $f(\Delta, |\mathrm{y}|, k) \cdot (\log \|\mathrm{b}\|_\infty)^{\mathcal{O}(1)}$, the claim follows. □

5.3 Algorithm for n-fold integer programming

We are now ready to prove Theorem 24.

Proof of Theorem 24. Let $P = (C, \mathrm{a}, \langle D_i, \mathrm{b}_i, \mathrm{c}_i \colon i \in I\rangle)$ be the given program. By adding some dummy variables and constraints if necessary, we may assume that all vectors b_i are non-zero. The first step of the algorithm is to construct a new program P', equivalent to P, in which all the right-hand sides b_i will be bounded in the ℓ_∞-norm. This essentially boils down to applying Lemma 36 to every b_i as follows.

For every $i \in I$, apply Lemma 36 to matrix D_i and vector b_i, yielding a faithful decomposition $\mathcal{B}_i$ of b_i with respect to D_i of order at most $\Xi := g(\Delta, |\mathrm{t}|)$. Obtain a new uniform n-fold program $P' = (C, \mathrm{a}, \langle D_j, \mathrm{b}_j, \mathrm{c}_j \colon j \in J\rangle)$ by replacing the single vector b_i with the collection of vectors $\mathcal{B}_i$, for each $i \in I$. Thus every index $j \in J$ originates in some index $i \in I$, and for all indices j originating in i we put $D_j = D_i$ and $\mathrm{c}_j = \mathrm{c}_i$.

Since collections $\mathcal{B}_i$ were faithful decompositions of vectors b_i with respect to D_i, for all $i \in I$, it follows that the program P' is equivalent to P in terms of feasibility and the minimum attainable value of the optimization goal. This is because every solution to P can be decomposed into a solution to P' with the same optimization goal value using faithfulness of the decompositions $\mathcal{B}_i$, and also every solution to P' can be naturally composed into a solution to P with the same value.

We note that the program P' is not computed by the algorithm explicitly, but in a high-multiplicity form. That is, decompositions $\mathcal{B}_i$ are output by the algorithm of Lemma 36 by providing every vector present in $\mathcal{B}_i$ together with its multiplicity in $\mathcal{B}_i$. Consequently, we may describe P' concisely by writing the multiplicity of every brick type present in P', where the brick type of an index $j \in J$ is defined by

- the diagonal matrix D_j;
- the right-hand side b_j; and
- the optimization vector c_j.

In other words, indices $j, j' \in J$ with $D_j = D_{j'}$, $\mathrm{b}_j = \mathrm{b}_{j'}$, and $\mathrm{c}_j = \mathrm{c}_{j'}$ are considered to have the same brick type. Note that there are at most $(2\Delta + 1)^{|\mathrm{y}|\cdot|\mathrm{t}|}$ different diagonal matrices D_j, at most $(2\Xi + 1)^{|\mathrm{t}|}$ different right-hand sides b_j, and at most $|I| \leqslant \|P\|$ different optimization vectors c_j, hence the total number of possible brick types is bounded $h(\Delta, |\mathrm{y}|, |\mathrm{t}|) \cdot \|P\|$, for some computable function h. For each such brick type, we store one positive integer of bitsize bounded by $\|P\|$ describing the multiplicity.

We now aim to solve the n-fold integer program P' efficiently by formulating it as another integer program M, and then relaxing M to a mixed integer program M' with a bounded number of integral variables. Intuitively, M will determine multiplicities in which solutions to individual bricks are taken, and assign them to the blocks in an optimal manner. The solutions to bricks will not be guessed explicitly, but through their decompositions provided by Lemma 26.

To formulate M, we need first some notation regarding P'. Let

- $\mathsf{Diag} = \{-\Delta, \ldots, \Delta\}^{\mathrm{y}\times\mathrm{t}}$ be the set of all possible diagonal blocks; and
- $\mathsf{RHS} = \{-\Xi, \ldots, \Xi\}^{\mathrm{t}}$ be the set of all possible right-hand sides.

Also, I is the index set of the collection $\langle \mathrm{c}_i \colon i \in I\rangle$ consisting of all possible optimization vectors. Thus, the set of all possible brick types that may be

present in P' is $\mathsf{Types} \coloneqq \mathsf{RHS} \times \mathsf{Diag} \times I$, and we assume that P' is stored by providing, for each type $(D, \mathrm{b}, i) \in \mathsf{Types}$, the multiplicity $\mathsf{count}[D, \mathrm{b}, i]$ with which the type (D, b, i) appears in P'.

For $D \in \mathsf{Diag}$ and $\mathrm{b} \in \mathsf{RHS}$, we let $\mathsf{Base}[D, \mathrm{b}]$ be the set of all base solutions for D and b: vectors $\widehat{\mathrm{w}} \in \mathbb{Z}^{\mathrm{y}}_{\geqslant 0}$ such that $D\widehat{\mathrm{w}} = \mathrm{b}$ and $\|\widehat{\mathrm{w}}\| \leqslant (2|\mathrm{t}|(\Delta + \Xi) + 1)^{|\mathrm{t}|}$. Recall that by Lemma 26, every nonnegative integer solution w to $D\mathrm{w} = \mathrm{b}$ can be decomposed as

$$\mathrm{w} = \widehat{\mathrm{w}} + \sum_{\mathrm{g} \in \mathcal{G}} \mathrm{g}, \tag{17}$$

where $\widehat{\mathrm{w}} \in \mathsf{Base}[D, \mathrm{b}]$ and $\mathcal{G}$ is a multiset consisting of nonnegative vectors from the Graver basis. For convenience, denote $\mathsf{Gra}^+(D) \coloneqq \mathsf{Graver}(D) \cap \mathbb{Z}^{\mathrm{t}}_{\geqslant 0}$.

We may now define the variables of M. These will be:

- $\zeta^{D,\mathrm{b}}_{\widehat{\mathrm{w}}}$ for $D \in \mathsf{Diag}$, $\mathrm{b} \in \mathsf{RHS}$, and $\widehat{\mathrm{w}} \in \mathsf{Base}[D, \mathrm{b}]$, signifying how many times in total the base solution $\widehat{\mathrm{w}}$ will be taken for a brick with diagonal block D and right-hand side b;
- δ^D_{g} for $D \in \mathsf{Diag}$ and $\mathrm{g} \in \mathsf{Gra}^+(D)$, signifying how many times in total the Graver basis element g will appear in the decompositions (17) of solutions to bricks; and
- $\omega^{D,\mathrm{b},i}_{\widehat{\mathrm{w}}}$ for $D \in \mathsf{Diag}$, $\mathrm{b} \in \mathsf{RHS}$, $i \in I$, and $\widehat{\mathrm{w}} \in \mathsf{Base}[D, \mathrm{b}]$, signifying how many times the base solution $\widehat{\mathrm{w}}$ will be assigned to a brick of type (D, b, i).

Next, we define the constraints of M:

$$\sum_{D \in \mathsf{Diag}} \sum_{\mathrm{b} \in \mathsf{RHS}} \left(\sum_{\widehat{\mathrm{w}} \in \mathsf{Base}[D,\mathrm{b}]} \zeta^{D,\mathrm{b}}_{\widehat{\mathrm{w}}} \cdot C\widehat{\mathrm{w}} + \sum_{\mathrm{g} \in \mathsf{Gra}^+(D)} \delta^D_{\mathrm{g}} \cdot C\mathrm{g} \right) = \mathrm{a}; \tag{C1}$$

$$\sum_{i \in I} \omega^{D,\mathrm{b},i}_{\widehat{\mathrm{w}}} = \zeta^{D,\mathrm{b}}_{\widehat{\mathrm{w}}} \qquad \text{for all } D \in \mathsf{Diag}, \mathrm{b} \in \mathsf{RHS}, \text{ and } \widehat{\mathrm{w}} \in \mathsf{Base}[D, \mathrm{b}]; \tag{C2}$$

$$\sum_{\widehat{\mathrm{w}} \in \mathsf{Base}[D,\mathrm{b}]} \omega^{D,\mathrm{b},i}_{\widehat{\mathrm{w}}} = \mathsf{count}[D, \mathrm{b}, i] \qquad \text{for all } D \in \mathsf{Diag}, \mathrm{b} \in \mathsf{RHS}, \text{ and } i \in I; \tag{C3}$$

$$\zeta^{D,\mathrm{b}}_{\widehat{\mathrm{w}}} \in \mathbb{Z}_{\geqslant 0} \qquad \text{for all } D \in \mathsf{Diag}, \mathrm{b} \in \mathsf{RHS}, \text{ and } \widehat{\mathrm{w}} \in \mathsf{Base}[D, \mathrm{b}]; \tag{C4}$$

$$\delta^D_{\mathrm{g}} \in \mathbb{Z}_{\geqslant 0} \qquad \text{for all } D \in \mathsf{Diag} \text{ and } \mathrm{g} \in \mathsf{Gra}^+[D, \mathrm{b}]; \tag{C5}$$

$$\omega^{D,\mathrm{b},i}_{\widehat{\mathrm{w}}} \in \mathbb{Z}_{\geqslant 0} \qquad \text{for all } D \in \mathsf{Diag}, \mathrm{b} \in \mathsf{RHS}, \widehat{\mathrm{w}} \in \mathsf{Base}[D, \mathrm{b}], \text{ and } i \in I. \tag{C6}$$

Let us explain these constraints:

- Constraint (C1) corresponds to the linking constraints of the n-fold program P'.
- Constraint (C2) assures that the numbers $\omega^{D,\mathrm{b},i}_{\widehat{\mathrm{w}}}$ of base solutions $\widehat{\mathrm{w}} \in \mathsf{Base}[D, \mathrm{b}]$ assigned to bricks with different optimization vectors add up to the total number of such base solutions.
- In a similar vein, Constraint (C3) states that the number of bricks of a particular brick type matches the total number of suitable base solutions

assigned to them.
- Finally, constraints (C4), (C5), and (C6) ensure the integrality and non-negativity of our variables.

Last but not least, the objective function of M is to

$$\text{minimize} \quad \sum_{D\in\mathsf{Diag}} \sum_{\mathrm{b}\in\mathsf{RHS}} \sum_{\widehat{\mathrm{w}}\in\mathsf{Base}[D,\mathrm{b}]} \sum_{i\in I} \omega_{\widehat{\mathrm{w}}}^{D,\mathrm{b},i}\cdot\langle \mathrm{c}_i, \widehat{\mathrm{w}}\rangle + \sum_{D\in\mathsf{Diag}} \sum_{\mathrm{g}\in\mathsf{Gra}^+(D)} \delta_{\mathrm{g}}^{D}\cdot\mathsf{best}_{\mathrm{g}}^{D},$$

where for $D \in \mathsf{Diag}$ and $\mathrm{g} \in \mathsf{Gra}^+(D)$, we define $\mathsf{best}_{\mathrm{g}}^{D}$ to be the minimum of $\langle \mathrm{c}_i, \mathrm{g}\rangle$ over all $i \in I$ such that $D_i = D$. Here, the first summand is just the contribution of the base solutions to the optimization goal, while in the second summand we observe that every Graver basis element that appears in decompositions (17) can be assigned freely to any brick with the corresponding diagonal block D, hence we may greedily assign it to the brick where its contribution to the optimization goal is the smallest. Note that here we crucially use the assumption that the n-fold program P' is uniform, because we exploit the fact that assigning both the base solutions and the Graver basis elements to different bricks has always the same effect on the linking constraints.

From the construction it is clear that the integer programs P' and M are equivalent in terms of feasibility and the minimum attainable value of the optimization goal. However, the number of variables of M is too large to solve it directly: while the total number of variables $\zeta_{\widehat{\mathrm{w}}}^{D,\mathrm{b}}$ and δ_{g}^{D} is indeed bounded in terms of the parameters, this is not the case for the assignment variables $\omega_{\widehat{\mathrm{w}}}^{D,\mathrm{b},i}$, of which there may be as many as $\Omega(|I|)$.

To solve this difficulty, we consider a mixed program M' that differs from M by replacing (C6) with the following constraint:

$$\omega_{\widehat{\mathrm{w}}}^{D,\mathrm{b},i} \in \mathbb{R}_{\geqslant 0} \qquad \text{for all } D \in \mathsf{Diag}, \mathrm{b} \in \mathsf{RHS}, \widehat{\mathrm{w}} \in \mathsf{Base}[D,\mathrm{b}], \text{ and } i \in I. \tag{C7'}$$

That is, we relax variables $\omega_{\widehat{\mathrm{w}}}^{D,\mathrm{b},i}$ to be fractional. We now observe that M and M' in fact have the same optima, because after fixing the integral variables, the program M' encodes a weighted flow problem and therefore its constraint matrix is totally unimodular.

Claim 37. The minimum attainable optimization goal values of programs M and M' are equal.

Proof. As M' is a relaxation of M, it suffices to prove that M' has an optimum solution that is integral. Observe that after fixing the integral variables $\zeta_{\widehat{\mathrm{w}}}^{D,\mathrm{b}}$ and δ_{g}^{D}, the only remaining constraints are (C2) and (C3). These constraints encode that values $\omega_{\widehat{\mathrm{w}}}^{D,\mathrm{b},i}$ form a feasible flow in the following flow network:

- There is a source s, a sink t, and two set of vertices: $V_s \coloneqq \{(D,\mathrm{b},\widehat{\mathrm{w}})\colon D \in \mathsf{Diag}, \mathrm{b} \in \mathsf{RHS}, \widehat{\mathrm{w}} \in \mathsf{Base}[D,\mathrm{b}]\}$ and $V_t \coloneqq \{(D,\mathrm{b},i)\colon D \in \mathsf{Diag}, \mathrm{b} \in \mathsf{RHS}, i \in I\}$.
- For every $(D,\mathrm{b},\widehat{\mathrm{w}}) \in V_s$, there is an arc from s to $(D,\mathrm{b},\widehat{\mathrm{w}})$ with capacity $\zeta_{\widehat{\mathrm{w}}}^{D,\mathrm{b}}$.

- For every $(D, \mathrm{b}, i) \in V_t$, there is an arc from (D, b, i) to t with capacity $\mathsf{count}[D, \mathrm{b}, i]$.
- For every $D \in \mathsf{Diag}$, $\mathrm{b} \in \mathsf{RHS}$, $\widehat{\mathrm{w}} \in \mathsf{Base}[D, \mathrm{b}]$, and $i \in I$, there is an arc from $(D, \mathrm{b}, \widehat{\mathrm{w}})$ to (D, b, i) with infinite capacity.

It is well known that constraint matrices of flow problems are totally unimodular, hence so is the matrix of program M' after fixing any evaluation of the integral variables. Vertices of polyhedra defined by totally unimodular matrices are integral, hence after fixing the integral variables of M', the fractional variables can be always made integral without increasing the optimization goal value. This shows that M' always has an optimum solution that is integral. □

Consequently, to find the optimum value for the initial n-fold program P, it suffices to solve the mixed program M'. As M' can be constructed in time $f(\Delta, |\mathrm{y}|, |\mathrm{t}|) \cdot \|P\|^{\mathcal{O}(1)}$ and has $f(\Delta, |\mathrm{y}|, |\mathrm{t}|)$ integral variables, for some computable function f, we may apply any fixed-parameter algorithm for solving mixed programs, for instance that of Lenstra [43], to solve M' within the promised running time bounds.

Finally, let us remark that while the presented procedure outputs only the optimum value for P, one can actually find an optimum solution to P by (i) finding an optimum mixed solution to M', (ii) finding an optimum integer solution to M' by solving the fractional part integrally using e.g. the flow formulation constructed in Claim 37, and (iii) translating the obtained optimum solution to M' back to an optimum solution to P. We leave the details to the reader. □

References

[1] Divesh Aggarwal, Antoine Joux, Miklos Santha, and Karol Węgrzycki. Polynomial time algorithms for Integer Programming and Unbounded Subset Sum in the total regime. ArXiv Preprint, abs/2407.05435, 2024. URL `https://doi.org/10.48550/arXiv.2407.05435`.

[2] Iskander Aliev and Martin Henk. Feasibility of integer knapsacks. SIAM J. Optim., 20(6):2978–2993, 2010. doi: 10.1137/090778043. URL `https://doi.org/10.1137/090778043`.

[3] Matthias Aschenbrenner and Raymond Hemmecke. Finiteness theorems in stochastic integer programming. Found. Comput. Math., 7(2):183–227, 2007. doi: 10.1007/s10208-005-0174-1. URL `https://doi.org/10.1007/s10208-005-0174-1`.

[4] Eleonore Bach, Friedrich Eisenbrand, Thomas Rothvoss, and Robert Weismantel. Forall-exist statements in pseudopolynomial time. In 36th Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2025, pages 2225–2233. SIAM, 2025. doi: 10.1137/1.9781611978322.73. URL `https://doi.org/10.1137/1.9781611978322.73`.

[5] Marcin Briański, Martin Koutecký, Daniel Král', Kristýna Pekárková, and Felix Schröder. Characterization of matrices with bounded Graver bases and depth parameters and applications to integer programming. Math. Program., 208(1):497–531, 2024. doi: 10.1007/S10107-023-02048-X. URL https://doi.org/10.1007/s10107-023-02048-x.

[6] Timothy F. N. Chan, Jacob W. Cooper, Martin Koutecký, Daniel Král, and Kristýna Pekárková. Matrices of optimal tree-depth and a row-invariant parameterized algorithm for integer programming. SIAM J. Comput., 51 (3):664–700, 2022. doi: 10.1137/20m1353502. URL https://doi.org/10.1137/20m1353502.

[7] Hua Chen, Lin Chen, and Guochuan Zhang. Block-structured integer programming: Can we parameterize without the largest coefficient? Discret. Optim., 46:100743, 2022. doi: 10.1016/j.disopt.2022.100743. URL https://doi.org/10.1016/j.disopt.2022.100743.

[8] Hua Chen, Lin Chen, and Guochuan Zhang. FPT algorithms for a special block-structured integer program with applications in scheduling. Math. Program., 208(1):463–496, 2024. doi: 10.1007/S10107-023-02046-Z. URL https://doi.org/10.1007/s10107-023-02046-z.

[9] Lin Chen and Dániel Marx. Covering a tree with rooted subtrees — parameterized and approximation algorithms. In 29th Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2018, pages 2801–2820. SIAM, 2018. doi: 10.1137/1.9781611975031.178. URL https://doi.org/10.1137/1.9781611975031.178.

[10] Lin Chen, Dániel Marx, Deshi Ye, and Guochuan Zhang. Parameterized and approximation results for scheduling with a low rank processing time matrix. In 34th Symposium on Theoretical Aspects of Computer Science, STACS 2017, volume 66 of LIPIcs, pages 22:1–22:14. Schloss Dagstuhl — Leibniz-Zentrum für Informatik, 2017. doi: 10.4230/LIPIcs.STACS.2017.22. URL https://doi.org/10.4230/LIPIcs.STACS.2017.22.

[11] Lin Chen, Martin Koutecký, Lei Xu, and Weidong Shi. New bounds on augmenting steps of block-structured integer programs. In 28th Annual European Symposium on Algorithms, ESA 2020, volume 173 of LIPIcs, pages 33:1–33:19. Schloss Dagstuhl — Leibniz-Zentrum für Informatik, 2020. doi: 10.4230/LIPIcs.ESA.2020.33. URL https://doi.org/10.4230/LIPIcs.ESA.2020.33.

[12] Jana Cslovjecsek, Friedrich Eisenbrand, Christoph Hunkenschröder, Lars Rohwedder, and Robert Weismantel. Block-structured integer and linear programming in strongly polynomial and near linear time. In 32nd Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2021, pages 1666–1681. SIAM, 2021. doi: 10.1137/1.9781611976465.101. URL https://doi.org/10.1137/1.9781611976465.101.

[13] Jana Cslovjecsek, Friedrich Eisenbrand, Michał Pilipczuk, Moritz Venzin, and Robert Weismantel. Efficient sequential and parallel algorithms for multistage stochastic integer programming using proximity. In 29th Annual European Symposium on Algorithms, ESA 2021, volume 204 of LIPIcs, pages 33:1–33:14. Schloss Dagstuhl — Leibniz-Zentrum für Informatik, 2021. doi: 10.4230/LIPIcs.ESA.2021.33. URL https://doi.org/10.4230/LIPIcs.ESA.2021.33.

[14] Daniel Dadush. Integer Programming, Lattice Algorithms, and Deterministic Volume Estimation. PhD thesis, Georgia Institute of Technology, USA, 2012.

[15] Daniel Dadush, Friedrich Eisenbrand, and Thomas Rothvoss. From approximate to exact integer programming. Math. Program., 210(1):223–241, 2025. doi: 10.1007/S10107-024-02084-1. URL https://doi.org/10.1007/s10107-024-02084-1.

[16] Jesús A. De Loera, Raymond Hemmecke, and Matthias Köppe. Algebraic and Geometric Ideas in the Theory of Discrete Optimization, volume 14 of MOS-SIAM Series on Optimization. SIAM, 2013. ISBN 978-1-61197-243-6. doi: 10.1137/1.9781611972443.

[17] Jesús A. De Loera, Raymond Hemmecke, Shmuel Onn, and Robert Weismantel. N-fold integer programming. Discrete Optimization, 5(2):231–241, 2008. ISSN 1572-5286. doi: https://doi.org/10.1016/j.disopt.2006.06.006. In Memory of George B. Dantzig.

[18] Friedrich Eisenbrand and Gennady Shmonin. Parametric integer programming in fixed dimension. Math. Oper. Res., 33(4):839–850, 2008. doi: 10.1287/MOOR.1080.0320.

[19] Friedrich Eisenbrand, Christoph Hunkenschröder, and Kim-Manuel Klein. Faster algorithms for integer programs with block structure. In 45th International Colloquium on Automata, Languages, and Programming, ICALP 2018, volume 107 of LIPIcs, pages 49:1–49:13. Schloss Dagstuhl — Leibniz-Zentrum für Informatik, 2018. doi: 10.4230/LIPIcs.ICALP.2018.49. URL https://doi.org/10.4230/LIPIcs.ICALP.2018.49.

[20] Friedrich Eisenbrand, Christoph Hunkenschröder, Kim-Manuel Klein, Martin Koutecký, Asaf Levin, and Shmuel Onn. An algorithmic theory of integer programming. ArXiv Preprint, abs/1904.01361, 2019. URL http://arxiv.org/abs/1904.01361.

[21] András Frank and Éva Tardos. An application of simultaneous diophantine approximation in combinatorial optimization. Combinatorica, 7(1):49–65, 1987. doi: 10.1007/BF02579200.

[22] Tomáš Gavenčiak, Martin Koutecký, and Dušan Knop. Integer programming in parameterized complexity: Five miniatures. Discret. Optim., 44(Part):100596, 2022. doi: 10.1016/j.disopt.2020.100596. URL https://doi.org/10.1016/j.disopt.2020.100596.

[23] Raymond Hemmecke, Shmuel Onn, and Lyubov Romanchuk. n-fold integer programming in cubic time. Math. Program., 137(1-2):325–341, 2013. doi: 10.1007/s10107-011-0490-y. URL https://doi.org/10.1007/s10107-011-0490-y.

[24] Raymond Hemmecke, Matthias Köppe, and Robert Weismantel. Graver basis and proximity techniques for block-structured separable convex integer minimization problems. Math. Program., 145(1-2):1–18, 2014. doi: 10.1007/s10107-013-0638-z.

[25] Klaus Jansen, Alexandra Lassota, and Lars Rohwedder. Near-linear time algorithm for n-fold ILPs via color coding. SIAM J. Discret. Math., 34(4): 2282–2299, 2020. doi: 10.1137/19M1303873. URL https://doi.org/10.1137/19M1303873.

[26] Klaus Jansen, Kim-Manuel Klein, Marten Maack, and Malin Rau. Empowering the configuration-IP: new PTAS results for scheduling with setup times. Math. Program., 195(1):367–401, 2022. doi: 10.1007/s10107-021-01694-3. URL https://doi.org/10.1007/s10107-021-01694-3.

[27] Klaus Jansen, Kim-Manuel Klein, and Alexandra Lassota. The double exponential runtime is tight for 2-stage stochastic ILPs. Math. Program., 197(2):1145–1172, 2023. doi: 10.1007/s10107-022-01837-0. URL https://doi.org/10.1007/s10107-022-01837-0.

[28] Ravi Kannan. Minkowski's convex body theorem and integer programming. Mathematics of Operations Research, 12(3):415–440, 1987. doi: 10.1287/MOOR.12.3.415.

[29] Kim-Manuel Klein. About the complexity of two-stage stochastic IPs. Math. Program., 192(1):319–337, 2022. doi: 10.1007/S10107-021-01698-Z.

[30] Kim-Manuel Klein and Janina Reuter. Collapsing the tower — On the complexity of multistage stochastic IPs. ACM Trans. Algorithms, 20(3):22, 2024. doi: 10.1145/3604554. URL https://doi.org/10.1145/3604554.

[31] Dušan Knop and Martin Koutecký. Scheduling meets n-fold integer programming. J. Sched., 21(5):493–503, 2018. doi: 10.1007/s10951-017-0550-0. URL https://doi.org/10.1007/s10951-017-0550-0.

[32] Dušan Knop and Martin Koutecký. Scheduling kernels via configuration LP. In 30th Annual European Symposium on Algorithms, ESA 2022, volume

244 of LIPIcs, pages 73:1–73:15. Schloss Dagstuhl — Leibniz-Zentrum für Informatik, 2022. doi: 10.4230/LIPIcs.ESA.2022.73. URL https://doi.org/10.4230/LIPIcs.ESA.2022.73.

[33] Dušan Knop, Martin Koutecký, and Matthias Mnich. Combinatorial n-fold integer programming and applications. Math. Program., 184(1):1–34, 2020. doi: 10.1007/s10107-019-01402-2. URL https://doi.org/10.1007/s10107-019-01402-2.

[34] Dušan Knop, Martin Koutecký, and Matthias Mnich. Voting and bribing in single-exponential time. ACM Trans. Economics and Comput., 8(3): 12:1–12:28, 2020. doi: 10.1145/3396855. URL https://doi.org/10.1145/3396855.

[35] Dušan Knop, Michał Pilipczuk, and Marcin Wrochna. Tight complexity lower bounds for integer linear programming with few constraints. ACM Trans. Comput. Theory, 12(3):19:1–19:19, 2020. doi: 10.1145/3397484. URL https://doi.org/10.1145/3397484.

[36] Dušan Knop, Martin Koutecký, Asaf Levin, Matthias Mnich, and Shmuel Onn. Parameterized complexity of configuration integer programs. Oper. Res. Lett., 49(6):908–913, 2021. doi: 10.1016/j.orl.2021.11.005. URL https://doi.org/10.1016/j.orl.2021.11.005.

[37] Dušan Knop, Martin Koutecký, Asaf Levin, Matthias Mnich, and Shmuel Onn. High-multiplicity N-fold IP via configuration LP. Math. Program., 200(1):199–227, 2023. doi: 10.1007/S10107-022-01882-9.

[38] Martin Koutecký and Nimrod Talmon. Multi-party campaigning. ArXiv Preprint, abs/2005.04455, 2020. URL https://arxiv.org/abs/2005.04455.

[39] Martin Koutecký and Nimrod Talmon. Multi-party campaigning. In 35th AAAI Conference on Artificial Intelligence, AAAI 2021, pages 5506–5513. AAAI Press, 2021. doi: 10.1609/AAAI.V35I6.16693.

[40] Martin Koutecký and Johannes Zink. Complexity of scheduling few types of jobs on related and unrelated machines. In 31st International Symposium on Algorithms and Computation, ISAAC 2020, volume 181 of LIPIcs, pages 18:1–18:17. Schloss Dagstuhl — Leibniz-Zentrum für Informatik, 2020. doi: 10.4230/LIPIcs.ISAAC.2020.18. URL https://doi.org/10.4230/LIPIcs.ISAAC.2020.18.

[41] Martin Koutecký, Asaf Levin, and Shmuel Onn. A parameterized strongly polynomial algorithm for block structured integer programs. In 45th International Colloquium on Automata, Languages, and Programming, ICALP 2018, volume 107 of LIPIcs, pages 85:1–85:14. Schloss Dagstuhl — Leibniz-Zentrum für Informatik, 2018. doi: 10.4230/LIPIcs.ICALP.2018.85. URL https://doi.org/10.4230/LIPIcs.ICALP.2018.85.

[42] J. C. Lagarias. The computational complexity of simultaneous diophantine approximation problems. SIAM J. Comput., 14(1):196–209, 1985. doi: 10.1137/0214016.

[43] H. W. Lenstra. Integer programming with a fixed number of variables. Mathematics of Operations Research, 8(4):538–548, 1983. ISSN 0364765X, 15265471.

[44] Loic Pottier. Minimal solutions of linear diophantine systems: Bounds and algorithms. In 4th International Conference on Rewriting Techniques and Applications, RTA-91, volume 488 of Lecture Notes in Computer Science, pages 162–173. Springer, 1991. doi: 10.1007/3-540-53904-2_94. URL https://doi.org/10.1007/3-540-53904-2_94.

[45] Mojżesz Presburger. Über die Vollständigkeit eines gewissen Systems der Arithmetik ganzer Zahlen, in welchem die Addition als einzige Operation hervortritt. In Comptes Rendus du I congrès de Mathématiciens des Pays Slaves, pages 92–101, 1929.

[46] Victor Reis and Thomas Rothvoss. The subspace flatness conjecture and faster integer programming. In 64th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2023, pages 974–988. IEEE, 2023. doi: 10.1109/FOCS57990.2023.00060. URL https://doi.org/10.1109/FOCS57990.2023.00060.

[47] R. Schultz, L. Stougie, and M. H. van der Vlerk. Two-stage stochastic integer programming: a survey. Statistica Neerlandica, 50(3):404–416, 1996. doi: 10.1111/j.1467-9574.1996.tb01506.x.

[48] Hermann Weyl. Elementare Theorie der konvexen Polyeder. Commentarii Mathematici Helveticii, 7:290–306, 1935.

A Proof of Observation 3

In this section, we prove Observation 3, restated below for convenience.

Observation 3. (Restated) Suppose the feasibility problem for uniform 4-block programs can be solved in time $f(k, \Delta) \cdot \|P\|^{\mathcal{O}(1)}$ for some computable function f, where k is the dimension of every block and Δ is the maximum absolute value of any entry in the constraint matrix. Then the feasibility problem for uniform 4-block programs can be also solved in time $g(k, \max_i \|D_i\|_\infty) \cdot \|P\|^{\mathcal{O}(1)}$ for some computable function g under the assumption that all the absolute values of the entries in matrices A, B, C are bounded by n.

Proof. Let P be the input program:

$$\mathrm{x} \in \mathbb{Z}^k_{\geqslant 0},\ \mathrm{y}_t \in \mathbb{Z}^k_{\geqslant 0},$$

$$B\mathrm{x} + \sum_{t=1}^{n} C_t \mathrm{y}_t = \mathrm{a}, \qquad \text{and} \tag{18}$$

$$A_t\mathrm{x} + D_t\mathrm{y}_t = \mathrm{b}_t \qquad \text{for all } t = 1, 2, \ldots, n, \tag{19}$$

where A, B, C, D_i are integer $k \times k$ matrices and a, b_i are integer vectors of length k. Denote $A = [a_{ij}]_{i,j\in[k]}$, $B = [b_{ij}]_{i,j\in[k]}$, and $C = [c_{ij}]_{i,j\in[k]}$, and recall that we assume that all entries in A, B, C have absolute values bounded by n. Also, let $\mathrm{a} = (a_1, \ldots, a_k)^\intercal$, $\mathrm{x} = (x_1, \ldots, x_k)^\intercal$, and $\mathrm{y}_t = (y_{t,1}, \ldots, y_{t,k})^\intercal$, for $t = 1, \ldots, n$. Variables x will be called global variables and variables y_t will be called local variables. Similarly, constraints (18) will be called linking, and constraints (19) will be called local. Local variables and local constraints come in t groups, each consisting of k variables/constraints.

We will gradually modify the program P so that at the end, in the modified blocks A, B, C all the coefficients will belong to $\{-1, 0, 1\}$. During the modification, we do not insist on all blocks being always square matrices; this can be always fixed at the end by adding some zero rows or zero columns.

First, we reduce large entries in C. Note that the ith linking constraint takes the form

$$\sum_{j=1}^{k} b_{ij}\, x_j + \sum_{t=1}^{n} \sum_{j=1}^{k} c_{ij}\, y_{t,j} = a_i,$$

or equivalently,

$$\sum_{j=1}^{k} b_{ij}\, x_j + \sum_{j=1}^{k} c_{ij} \sum_{t=1}^{n} y_{t,j} = a_i.$$

Hence, for every pair $(i, j) \in [k] \times [k]$ we introduce a new global variable z_{ij} together with the constraint

$$-z_{ij} + \sum_{t=1}^{n} y_{t,j} = 0,$$

and in the ith linking constraint, we replace the summand $c_{ij} \sum_{t=1}^{n} y_{t,j}$ with the summand $c_{ij} z_{ij}$. It is easy to see that after this modification, only $\{0, 1\}$ entries will appear in the blocks C. This comes at the price of increasing the number of global variables and the number of linking constraints by k^2, and possibly adding some large (but bounded by n in absolute value) entries to the block B.

Next, we remove large entries in A. Suppose then $a_{ij} > 1$ for some $i, j \in [k]$; recall that we assume also that $a_{ij} \leqslant n$. In program P, entry a_{ij} occurs in summands of the form $a_{ij} x_j$; every group of local constraints contains one such summand. Therefore, the idea is to (i) copy the variable x_j to a_i new local variables, (ii) use one new linking constraint to define a new global variable z'_{ij} equal to the sum of the copies, and hence equal to $a_{ij} x_j$, and (iii) replace each usage of the summand $a_{ij} x_j$ in local constraints with just the variable z'_{ij}. This idea can be executed as follows:

(i) For every $t = 1, \ldots, n$, introduce two new local variables $p_{ij,t}, p'_{ij,t}$ together with two new local constraints:

$$-x_j + p'_{ij,t} = 0; \qquad \text{and}$$
$$p_{ij,t} - p'_{ij,t} = 0 \text{ when } t \leqslant a_{ij} \qquad \text{and} \qquad p_{ij,t} = 0 \text{ when } t > a_{ij}.$$

(ii) Add a new global variable z'_{ij} together with a new linking constraint

$$-z'_{ij} + \sum_{t=1}^{n} p_{ij,t} = 0.$$

(iii) Replace every usage of summand $a_{ij} x_{ij}$ with variable z'_{ij} in every local constraint within P.

Entries $a_{ij} \in \{-n, \ldots, -2\}$ can be removed similarly: we perform the above construction for $|a_{ij}|$, and then replace each summand $a_{ij} x_{ij}$ with $-z'_{ij}$ instead of z'_{ij}. Entries with large absolute values in B can be removed in exactly the same way.

It is straightforward to see that by applying all the modifications presented above, we obtain a uniform 4-block program with $\mathcal{O}(k^2)$ global variables, $\mathcal{O}(k^2)$ linking constraints, $\mathcal{O}(k^2)$ local variables in each group of local variables, and $\mathcal{O}(k^2)$ local constraints in each group of local constraints. Moreover, all entries in the blocks A, B, C belong to $\{-1, 0, 1\}$, while blocks D_i got modified only by adding some $\{-1, 0, 1\}$ entries. Hence, we can just solve the modified program using the assumed fixed-parameter algorithm for feasibility of uniform 4-block programs parameterized by the dimensions of the block and the maximum absolute value of any entry appearing in the constraint matrix. □

B Hardness of two-stage stochastic integer programming with large diagonal entries

We conclude our discussion of two-stage stochastic integer programming by a short argument showing that it is indeed necessary to include $\Delta = \max_{i \in I} \|D_i\|_\infty$

among the parameters. This is because when Δ is unbounded the problem becomes strongly NP-hard already for blocks of size $k = 16$, as we prove below. This rules out even a running time of the form $f(k) \cdot (\|P\| + \Delta)^{\mathcal{O}(1)}$, and shows that the dependence on Δ needs to be superpolynomial.

We give a reduction from the 3-SAT problem. Let N denote the number of variables and M the number of clauses in the input formula. The reduction creates an instance of Two-Stage Stochastic ILP Feasability with only one global variable $x \in \mathbb{Z}_{\geqslant 0}$, whose role is to encode a satisfying 3-SAT assignment. Let $p_1, p_2, \ldots, p_N$ be the first N primes. In our encoding, $p_i \mid x$ if and only if the i-th SAT variable is set to false. The IP instance contains M diagonal blocks, each corresponding to a single clause.

We use a certain gadget to construct the blocks. The gadget consists of five ILP variables, named $a, b, c, d, e \in \mathbb{Z}_{\geqslant 0}$, and three constraints that involve those five variables as well as the global variable x. Let p be a fixed prime (not an ILP variable). The key properties of the gadget are:

- if $p \mid x$, then every feasible assignment to the five local variables has $b = 0$;
- if $p \nmid x$, then every feasible assignment to the five local variables has $b = 1$; and
- for every $x \in \mathbb{Z}_{\geqslant 0}$ there exists a feasible assignment to the five local variables.

We leave it to the reader to verify that the following gadget satisfies the desired properties.

$$\begin{aligned} -x + p \cdot a + b + c &= 0 \\ (p-2) \cdot b - c - d &= 0 \qquad \text{(equivalent to } c \leqslant (p-2) \cdot b\text{)} \\ b + e &= 1 \qquad \text{(equivalent to } b \leqslant 1\text{)} \end{aligned}$$

Each block consists of three such independent gadgets, instantiated for the three primes corresponding to the three SAT variables involved in the corresponding clause. Moreover, each block contains one additional constraint (and one additional local variable) ensuring that the three b-variables (or their negations in case of negative literals) sum up to at least one (minus the number of negative literals in the clause), which corresponds to the clause being satisfied.

Summarizing, the resulting two-stage stochastic ILP instance is:

$$D_i = \begin{bmatrix} p_{v_{i,1}} & 1 & 1 & & & & & & & & & & & & \\ & p_{v_{i,1}}-2 & -1 & -1 & & & & & & & & & & & \\ & 1 & & & 1 & & & & & & & & & & \\ & & & & & p_{v_{i,2}} & 1 & 1 & & & & & & & \\ & & & & & & p_{v_{i,2}}-2 & -1 & -1 & & & & & & \\ & & & & & & 1 & & & 1 & & & & & \\ & & & & & & & & & & p_{v_{i,3}} & 1 & 1 & & \\ & & & & & & & & & & & p_{v_{i,3}}-2 & -1 & -1 & \\ & & & & & & & & & & & 1 & & & 1 \\ & t_{i,1} & & & & & t_{i,2} & & & & & t_{i,3} & & & -1 \end{bmatrix},$$

$$A_i = \begin{bmatrix} -1 \\ 0 \\ 0 \\ -1 \\ 0 \\ 0 \\ -1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathrm{b}_i = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 - \ell_i \end{bmatrix}, \quad \text{for } i \in [M],$$

where, for every for $i \in [M]$ and $j \in \{1, 2, 3\}$, $v_{i,j}$ denotes the index of the variable in the j-th literal in the i-th clause, $t_{i,j} = 1$ if this is a positive literal and $t_{i,j} = -1$ otherwise, and ℓ_i equals the number of negative literals in the i-th clause, i.e., $\ell_i = (3 - (t_{i,1} + t_{i,2} + t_{i,3}))/2$.

From the preceding discussion it follows that the obtained integer program is feasible if and only if the input 3-SAT formula is satisfiable. Each A_i is a 10×1 matrix, each D_i is a 10×16 matrix, and we have $\max_{i \in I} \|D_i\|_\infty \leqslant p_N = \mathcal{O}(N \log N)$ by the Prime Number Theorem. We conclude that the two-stage stochastic integer programming feasibility problem is strongly NP-hard already for $k = 16$.